

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH**



**XÂY DỰNG HỆ THỐNG TRỢ LÝ PHÁP LÝ AI CHO DOANH
NGHIỆP NHỎ VÀ VỪA**

Lê Gia Bảo

2351010016

Giảng viên hướng dẫn: **ThS. DƯƠNG HỮU THÀNH**

**ĐỒ ÁN NGÀNH
NGÀNH CÔNG NGHỆ THÔNG TIN**

TP. HỒ CHÍ MINH, 2026

**BỘ GIÁO DỤC VÀ ĐÀO TẠO
TRƯỜNG ĐẠI HỌC MỞ THÀNH PHỐ HỒ CHÍ MINH**



**XÂY DỰNG HỆ THỐNG TRỢ LÝ PHÁP LÝ AI CHO DOANH
NGHIỆP NHỎ VÀ VỪA**

Lê Gia Bảo

2351010016

Giảng viên hướng dẫn: ThS. DƯƠNG HỮU THÀNH

**ĐỒ ÁN NGÀNH
NGÀNH CÔNG NGHỆ THÔNG TIN**

TP. HỒ CHÍ MINH, 2026

LỜI CẢM ƠN

Em xin gửi lời cảm ơn đến Thầy ThS. Dương Hữu Thành, giảng viên hướng dẫn đồ án. Thầy đã góp ý sớm về phạm vi đề tài, về việc hệ thống phải chạy được khi thiếu khóa mô hình, và về việc mọi căn cứ trên giao diện phải mở được về Điều gốc. Những góp ý đó định hình cách em thiết kế hàng rào dẫn chiếu và cách đóng gói Docker.

Em cảm ơn quý Thầy Cô Khoa Công nghệ thông tin, Trường Đại học Mở Thành phố Hồ Chí Minh, đã giảng dạy các học phần nền để em đủ sức hoàn thành đồ án ngành.

Em cảm ơn gia đình đã tạo điều kiện học tập. Mọi thiếu sót trong báo cáo và trong mã nguồn thuộc về em.

Thành phố Hồ Chí Minh, năm 2026

Lê Gia Bảo

NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

TÓM TẮT

Đồ án xây dựng một ứng dụng web giúp doanh nghiệp nhỏ và vừa tra cứu và hỏi đáp pháp lý bằng tiếng Việt đời thường. Người dùng không cần nhớ số hiệu thông tư. Mỗi căn cứ hiện trên màn hình phải mở được đúng một Điều trong kho đã chuẩn hóa. Hệ thống hỗ trợ tra cứu, không thay thế luật sư hay đại lý thuế.

Kho gồm 38 văn bản quy phạm, 3.844 Điều, tập trung thuế, lao động, bảo hiểm xã hội, kế toán, hợp đồng và hỗ trợ DNNVV. Đơn vị lưu trữ là Điều, vì giới hành nghề dẫn chiếu theo Điều chứ không theo đoạn ký tự cắt sẵn. Phần hỏi đáp đi theo hướng truy hồi rồi mới sinh câu (RAG) [3]: tìm song song theo chữ (BM25) và theo nghĩa (vector), gộp hạng bằng RRF, lọc ngữ cảnh, rồi viết câu. Số Điều xuất hiện trong câu trả lời được đối chiếu với tập vừa tìm; số không thuộc tập này bị gỡ khỏi danh sách căn cứ.

Ba nghiệp vụ kèm theo: đọc văn bản theo cây Chương–Điều (tìm được khi gõ không dấu, nhưng đoạn trích vẫn còn dấu), soát xét hợp đồng PDF/DOCX/TXT, và lịch nhắc thuế–bảo hiểm–lao động–BCTC sinh theo hồ sơ từng công ty.

Ứng dụng dùng React, FastAPI, PostgreSQL và Chroma, đóng gói bằng Docker Compose. Khi thiếu khóa mô hình, đăng nhập, tra cứu và lịch vẫn chạy; chỉ hỏi đáp và soát xét báo chưa sẵn sàng.

Báo cáo có năm chương: giới thiệu, cơ sở lý thuyết, phân tích–thiết kế–hiện thực, thử nghiệm, kết luận.

ABSTRACT

This project is a multi-user web assistant for Vietnamese small and medium-sized enterprises. Users ask everyday questions and receive answers tied to articles that actually exist in a closed legal corpus. The system supports lookup and source checking; it does not replace a lawyer.

The corpus contains 38 normative texts and 3.844 articles, cut at article boundaries. Question answering follows retrieval-augmented generation [3]: BM25 and dense search run together, ranks are fused with RRF, context is filtered, then the answer is written. Article numbers that the model invents are stripped from the citation list if they were not in the retrieved set.

The same application also offers accent-insensitive full-text search with accented display, contract review for PDF/DOCX/TXT, and a compliance calendar generated from each firm's profile. The stack is React, FastAPI, PostgreSQL and Chroma, packaged with Docker Compose. Without a model API key, login, search and the calendar still work.

The report has five chapters: introduction, background, design and implementation, evaluation, and conclusion.

Mục lục

LỜI CẢM ƠN	I
NHẬN XÉT CỦA GIÁO VIÊN HƯỚNG DẪN	II
TÓM TẮT	III
ABSTRACT	IV
DANH MỤC TỪ VIẾT TẮT	X
DANH MỤC HÌNH VẼ	XI
DANH MỤC BẢNG	XII
1 Giới thiệu đề tài	1
1.1 Tổng quan tình hình liên quan đến đề tài	1
1.2 Bối cảnh pháp lý và đặc thù DNNVV	1
1.2.1 Doanh nghiệp nhỏ và vừa trong hệ thống pháp luật Việt Nam	1
1.2.2 Hệ thống văn bản quy phạm và thứ bậc áp dụng	2
1.2.3 Rủi ro tuân thủ đối với người không chuyên	2
1.2.4 Ranh giới trợ lý AI và dịch vụ pháp lý	2
1.3 Bài toán tin học đặt ra từ hiện trạng	2
1.4 Lý do lựa chọn đề tài	3
1.5 Mục tiêu đề tài	3
1.5.1 Mục tiêu tổng quát	3
1.5.2 Mục tiêu cụ thể	4
1.6 Cách tiếp cận và phương pháp	4
1.6.1 Hỏi đáp dạng open-book	4
1.6.2 Truy hồi lai	4
1.6.3 Quy trình phát triển	4
1.7 Đối tượng và phạm vi	5
1.7.1 Đối tượng	5
1.7.2 Phạm vi chức năng	5
1.7.3 Phạm vi dữ liệu và kỹ thuật	5

1.8	Tổng quan các giải pháp hiện có	5
1.9	Ý nghĩa của đề tài	6
1.10	Phương pháp nghiên cứu và công cụ	6
1.11	Bố cục báo cáo	6
2	Cơ sở lý thuyết	7
2.1	Xử lý ngôn ngữ tự nhiên trong bài toán hỏi đáp	7
2.2	Các khái niệm nền gắn với tiếng Việt và kho luật	7
2.2.1	Tách từ và hai lớp token	7
2.2.2	Từ phổ biến và độ hiếm	8
2.2.3	Người tìm, máy đọc	8
2.2.4	Bịa nội dung	8
2.2.5	Cửa sổ ngữ cảnh và nhiệt độ	8
2.2.6	Độ chính xác, độ phủ và cách đánh giá	9
2.3	Xác thực, ủy quyền và ba lớp phần mềm	9
2.4	Cấu trúc văn bản quy phạm pháp luật Việt Nam	9
2.4.1	Luật, nghị định, thông tư không nằm trên một trang	9
2.4.2	Điều là đơn vị cắt dữ liệu	9
2.4.3	Viện dẫn chéo	10
2.5	Truy hồi theo chữ: BM25	10
2.6	Truy hồi theo nghĩa: embedding và cosine	10
2.7	Gộp hai bảng xếp hạng: Reciprocal Rank Fusion	11
2.8	RAG, HyDE, lời nhắc mô hình và rủi ro bịa nguồn	12
2.8.1	RAG	12
2.8.2	HyDE	12
2.8.3	Lời nhắc mô hình, không tính chỉnh trọng số	12
2.8.4	Chỉ số F_2	13
2.9	Tìm chữ tiếng Việt trên PostgreSQL	13
2.10	Công nghệ hiện thực và lý do chọn	13
2.10.1	HTTP, JSON và CORS	13
2.10.2	Python	14
2.10.3	FastAPI	14
2.10.4	Pydantic	15
2.10.5	SQLAlchemy 2.0 và Alembic	15
2.10.6	PostgreSQL	15
2.10.7	ChromaDB	15
2.10.8	LangGraph	16
2.10.9	Uvicorn	16
2.10.10	Mô hình ngôn ngữ và Gemini	16

2.10.11 Underthesea	17
2.10.12 React	17
2.10.13 TypeScript	17
2.10.14 Vite	17
2.10.15 Tailwind CSS	17
2.10.16 TanStack Query	18
2.10.17 Zustand	18
2.10.18 React Router	18
2.10.19 Axios	18
2.10.20 nginx	19
2.10.21 Docker và Docker Compose	19
2.10.22 JWT	19
2.10.23 Argon2	19
2.10.24 Server-Sent Events	20
2.10.25 slowapi	20
2.10.26 Kiến trúc client–server và SPA	20
2.11 Chi tiết từng nút pipeline LangGraph	20
2.11.1 Nút analyze_intent	20
2.11.2 Nút prepare_retrieval_query	21
2.11.3 Nút retrieve	21
2.11.4 Nút rerank	21
2.11.5 Nút llm_filter	21
2.11.6 Nút generate_answer	22
2.11.7 Nút format_submission	22
2.12 So sánh phương án và lý do không chọn	22
2.12.1 REST và GraphQL	22
2.12.2 SSE, WebSocket và polling	23
2.12.3 JWT và phiên Redis	23
2.12.4 PostgreSQL và MongoDB	23
2.12.5 Chroma, FAISS và Pinecone	23
2.12.6 Next.js và Vite SPA	23
2.12.7 Redux Toolkit và Zustand	23
2.12.8 Celery/Redis và BackgroundTasks	23
2.12.9 Cookie HttpOnly và localStorage	23
2.12.10 Fine-tune và prompt	23
2.13 Bảng tổng kết: vai trò – thách thức – hướng xử lý	24
2.14 Công trình liên quan	24
3 Phân tích, thiết kế và hiện thực	25

3.1	Tổng quan chức năng từ góc nhìn người dùng	25
3.2	Yêu cầu	25
3.2.1	Chức năng	25
3.2.2	Phi chức năng	25
3.3	Kiến trúc ba lớp	25
3.4	Use case	27
3.4.1	Đặc tả bổ sung UC-05 và UC-09	27
3.5	Bảy bước hỏi đáp	27
3.6	Corpus	30
3.6.1	Nguồn ngữ liệu	31
3.7	Mô hình dữ liệu	31
3.7.1	Một số quyết định trên lược đồ dữ liệu	31
3.7.2	Chi tiết từng bảng và khóa ngoại	34
3.7.3	organizations và users	34
3.7.4	conversations và messages	34
3.7.5	legal_knowledge_records	34
3.7.6	compliance_rules và compliance_tasks	34
3.7.7	documents, contract_reviews, contract_findings	34
3.7.8	Nhật ký audit (nếu có trong schema)	35
3.8	Tìm chữ trên tiếng Việt	35
3.9	Luồng thời gian thực và lưu hội thoại	35
3.10	Hợp đồng	35
3.11	Lịch tuân thủ	36
3.12	Đăng nhập, phiên làm việc, bảo mật	37
3.13	Giao diện	38
3.14	Nhúng vector, cache BM25 và biến môi trường	38
3.15	Đóng gói ba dịch vụ	39
3.16	Tóm tắt hiện thực theo module	39
3.17	Thiết kế API, cấu hình và các khối hiện thực	39
3.17.1	API theo tài nguyên	39
3.17.2	Đăng ký và bootstrap quản trị	40
3.17.3	Cấu hình config.yaml	40
3.17.4	Tách chữ và soát xét hợp đồng	40
3.17.5	Giao diện từng trang	41
3.17.6	Nhật ký và chiến lược lỗi	41
3.18	Danh mục văn bản trong corpus	41
3.19	Mười hai nghĩa vụ tuân thủ và cách sinh lịch	44
3.19.1	VAT_MONTHLY — khai nộp GTGT theo tháng	44
3.19.2	VAT_QUARTERLY — khai nộp GTGT theo quý	44

3.19.3	CIT_PROVISIONAL — tạm nộp thuế TNDN theo quý	45
3.19.4	CIT_FINALIZATION — quyết toán TNDN năm	45
3.19.5	PIT_FINALIZATION — quyết toán TNCN năm	45
3.19.6	BUSINESS_LICENSE_FEE — lệ phí môn bài	45
3.19.7	SOCIAL_INSURANCE_MONTHLY — đóng BHXH hằng tháng . . .	45
3.19.8	LABOUR_REPORT_H1 và H2 — báo cáo tình hình lao động	45
3.19.9	LABOUR_SAFETY_REPORT — báo cáo an toàn vệ sinh lao động . .	45
3.19.10	FINANCIAL_STATEMENT — báo cáo tài chính năm	46
3.19.11	INVOICE_USAGE_CHECK — rà soát hóa đơn điện tử	46
3.19.12	Bảng tổng hợp rule	46
3.20	Quy trình chuẩn hóa ngữ liệu	47
4	Thử nghiệm và đánh giá	48
4.1	Môi trường thử nghiệm	48
4.2	Chạy tay trên cụm Docker	48
4.3	Kịch bản hỏi đáp	49
4.3.1	Câu trong phạm vi corpus	49
4.3.2	Câu không phải pháp luật	49
4.3.3	Câu ngoài corpus	49
4.4	Đánh giá module tra cứu và lịch tuân thủ	49
4.5	Đánh giá giao diện (định tính)	52
4.6	Đối chiếu mục tiêu Chương 1	52
4.7	Tiêu chí đánh giá định tính	52
4.7.1	Độ trễ, sẵn sàng và giới hạn đo	52
4.8	Kịch bản demo đề xuất khi bảo vệ	53
5	Kết luận và hướng phát triển	54
5.1	Kết quả đạt được	54
5.2	Hạn chế	54
5.3	Hướng phát triển	54
5.4	Kết luận	55
	TÀI LIỆU THAM KHẢO	56
	PHỤ LỤC	58

DANH MỤC TỪ VIẾT TẮT

Viết tắt	Đầy đủ		Dùng trong đề án để chỉ
API	Application Interface	Programming	Giao diện lập trình phía máy chủ
BHXX	Bảo hiểm xã hội		Nghĩa vụ đóng bảo hiểm của doanh nghiệp
BM25	Best Matching 25		Thuật toán chấm điểm khớp từ khi truy hồi Điều
BCTC	Báo cáo tài chính		Báo cáo năm doanh nghiệp phải nộp
CSDL	Cơ sở dữ liệu		PostgreSQL trong đề án
DNNVV	Doanh nghiệp nhỏ và vừa		Nhóm người dùng mục tiêu
FTS	Full-text search		Tìm kiếm toàn văn trên corpus
GTGT	Giá trị gia tăng		Thuế khai theo tháng hoặc theo quý
HyDE	Hypothetical Embeddings	Document	Sinh đoạn văn giả để truy hồi vector
JWT	JSON Web Token		Token xác thực, gồm access và refresh
LLM	Large Language Model		Mô hình sinh câu trả lời
ORM	Object-Relational Mapping		SQLAlchemy ánh xạ bảng thành đối tượng
RAG	Retrieval-Augmented Generation		Sinh câu trả lời sau khi truy hồi văn bản
RRF	Reciprocal Rank Fusion		Gộp hai bảng xếp hạng BM25 và vector
SPA	Single Page Application		Giao diện React một trang
SSE	Server-Sent Events		Đẩy token và tiến độ từ máy chủ xuống trình duyệt
TNCN	Thu nhập cá nhân		Thuế khấu trừ tiền lương
TNDN	Thu nhập doanh nghiệp		Thuế trên lợi nhuận
VBQPPL	Văn bản quy phạm pháp luật		Luật, nghị định, thông tư trong corpus

Danh sách hình vẽ

3.1	Ba lớp triển khai: trình duyệt → nginx → FastAPI; PostgreSQL, Chroma và Gemini nằm phía sau API.	26
3.2	Sơ đồ use case. Chủ / kế toán / nhân sự dùng bốn nghiệp vụ; quản trị thêm corpus và chạy hàng loạt câu hỏi.	28
3.3	Bảng nút LangGraph: nhánh SKIP thoát sớm; rerank tắt trên sản phẩm Gemini. .	30
3.4	Kiến trúc ba lớp: trình bày, ứng dụng, dữ liệu.	32
3.5	Lược đồ quan hệ PostgreSQL (12 bảng). Hội thoại thuộc user; hợp đồng và lịch thuộc organization. Corpus cắt theo Điều; laws là catalog đếm từ cùng law_id. .	33
3.6	Luồng hỏi đáp: ghi câu hỏi trước, stream SSE, ghi câu trả lời ở session CSDL riêng.	36
3.7	Luồng soát xét hợp đồng: trả 202 ngay, xử lý nền, UI poll trạng thái.	36
3.8	Máy trạng thái tài liệu: pending → ready → processing → done/failed.	36
4.1	Bố cục trang hỏi đáp: cột hội thoại, vết pipeline, câu trả lời và citation.	50
4.2	Bố cục trang tra cứu: cây Chương—Điều và tìm không dấu.	50
4.3	Bố cục trang hợp đồng: upload và bảng rủi ro.	51
4.4	Bố cục trang lịch tuân thủ: ô đếm và nhóm theo tháng.	51

Danh sách bảng

1.1	Đối chiếu cổng VBQPPL, chatbot không nguồn và hướng đồ án.	5
2.1	Công nghệ gắn vào sản phẩm.	14
2.2	Bảy nút LangGraph trên cấu hình sản phẩm.	22
2.3	Tổng kết khối truy hồi lại.	24
2.4	Tổng kết khối sinh câu và hậu kiểm trích dẫn.	24
2.5	Tổng kết khối nghiệp vụ DNNVV (hợp đồng + lịch).	24
3.1	Yêu cầu chức năng của hệ thống.	26
3.2	Ngưỡng phi chức năng của hệ thống.	26
3.3	Tác nhân.	27
3.4	Danh sách use case.	27
3.5	Đặc tả UC-03 — Hỏi đáp.	28
3.6	Đặc tả UC-07 — Soát xét hợp đồng.	29
3.7	Đặc tả UC-05 — Tra cứu và đọc văn bản.	29
3.8	Đặc tả UC-09 — Lịch tuân thủ.	29
3.9	Đếm nhanh theo loại và theo lĩnh vực (một văn bản vào đúng một lĩnh vực). . .	31
3.10	Các bảng nghiệp vụ và ghi chú hiện thực.	32
3.11	Bảng PostgreSQL chính và khóa ngoại then chốt.	35
3.12	Mười hai rule tuân thủ và điều kiện hồ sơ để rule được sinh.	37
3.13	Ánh xạ module code ↔ nghiệp vụ.	39
3.14	Danh mục 38 văn bản quy phạm trong corpus.	41
3.15	12 rule tuân thủ: tần suất và điều kiện lọc.	46
4.1	Cấu hình máy và cụm khi chạy thử.	48
4.2	Quan sát khi chạy cụm Docker trên máy làm đồ án.	48
4.3	Đối chiếu hồ sơ doanh nghiệp và task được sinh.	49
4.4	Đối chiếu bảy mục tiêu cụ thể với bằng chứng.	52

Chương 1. GIỚI THIỆU ĐỀ TÀI

1.1 Tổng quan tình hình liên quan đến đề tài

Doanh nghiệp nhỏ và vừa (DNNVV) tại Việt Nam phải thực hiện đồng thời nhiều nghĩa vụ pháp lý: thuế, lao động, bảo hiểm xã hội, kế toán và hợp đồng thương mại. Cổng thông tin văn bản quy phạm pháp luật của nhà nước cung cấp nguồn gốc đầy đủ và có tính pháp lý, nhưng cách tra cứu giả định người dùng đã biết số hiệu văn bản hoặc thuật ngữ chuyên môn. Trên thực tế, kế toán và chủ doanh nghiệp thường diễn đạt nhu cầu bằng ngôn ngữ đời thường, ví dụ “thử việc được bao lâu” hay “quý này nộp GTGT ngày nào”. Kết quả tìm theo từ khóa dễ bị quá rộng hoặc trượt mục tiêu, vì văn bản luật dùng cụm từ chuẩn hóa (“đơn phương chấm dứt hợp đồng lao động”) khác với cách nói thông dụng (“sa thải”).

Các hệ thống hội thoại dựa trên mô hình ngôn ngữ lớn trả lời tiếng Việt trôi chảy nhưng không bảo đảm dẫn chiếu đúng Điều. Khi đối chiếu với văn bản gốc, số Điều có thể không tồn tại hoặc nội dung bị trộn từ văn bản khác. Với nghĩa vụ kèm chế tài hành chính, câu trả lời sai nhưng mang tính khẳng định tạo rủi ro cao hơn so với việc không có câu trả lời.

Về kỹ thuật, kiến trúc truy hồi-rời-sinh (RAG) [3] buộc mô hình sinh câu trên ngữ cảnh đã truy hồi. Dense retrieval [4], BM25 [5], Reciprocal Rank Fusion [6] và HyDE [7] là các thành phần phổ biến để ghép pipeline hỏi đáp trên corpus hẹp. Công trình chatbot tư vấn tuyển sinh dùng RAG trên tiếng Việt [2] cho thấy hướng này khả thi khi kho tri thức có biên rõ. Đề án áp dụng các kỹ thuật trên vào kho VBQPPL phục vụ DNNVV, với đơn vị văn bản là Điều.

1.2 Bối cảnh pháp lý và đặc thù DNNVV

1.2.1 Doanh nghiệp nhỏ và vừa trong hệ thống pháp luật Việt Nam

Luật Hỗ trợ doanh nghiệp nhỏ và vừa số 04/2017/QH14 xác định DNNVV theo tiêu chí quy mô (lao động, doanh thu, nguồn vốn) gắn với ngành nghề, và gán một số chính sách hỗ trợ [8]. Đề án không tái lập toàn bộ tiêu chí phân loại để cấp “giấy chứng nhận DNNVV”. Hồ sơ tổ chức trong hệ thống chỉ lưu các tham số đủ để sinh lịch và mặc định chủ thể khi hỏi đáp: tên, mã số thuế, số lao động, kỳ kê khai GTGT.

Điểm thực tiễn: nghĩa vụ thuế, bảo hiểm, báo cáo lao động và hóa đơn không biến mất khi quy mô nhỏ. Bộ phận pháp chế riêng thì thường không có. Người dùng thật của đề án vì thế là chủ doanh nghiệp, kế toán dịch vụ hoặc nhân sự kiêm nhiệm — đối tượng hỏi bằng lời nói hàng ngày hơn là bằng số hiệu thông tư.

1.2.2 Hệ thống văn bản quy phạm và thứ bậc áp dụng

Văn bản do Quốc hội ban hành (luật, bộ luật) đặt nguyên tắc. Nghị định của Chính phủ quy định chi tiết. Thông tư của bộ hướng dẫn thực hiện. Câu hỏi thao tác (“nộp tờ khai ngày nào”, “hóa đơn gửi khi nào”) thường rơi xuống nghị định và thông tư, trong khi câu hỏi nguyên tắc (“có được đơn phương chấm dứt không”) rơi ở luật. Corpus đồ án cố ý chứa cả ba cấp trong các lĩnh vực đã chọn, tránh tình trạng trả lời đúng luật khung nhưng sai bước kê khai.

Hiệu lực theo thời gian là bài toán pháp điển riêng: văn bản mới thay văn bản cũ, điều khoản chuyển tiếp, văn bản hết hiệu lực còn nằm trên cổng. Đồ án lưu theo số hiệu, chưa gắn lịch sử hiệu lực từng Điều. Khi kho có hai luật cùng tên khác số (ví dụ quản lý thuế, thuế GTGT), cả hai đều tìm được; người dùng và người quản trị phải tự chọn bản đang áp dụng. Hạn chế này được nêu lại ở Chương 5, không giấu trong phần “đã hoàn thiện”.

1.2.3 Rủi ro tuân thủ đối với người không chuyên

Chậm nộp, sai tờ khai, thiếu báo cáo lao động có thể kéo theo tiền chậm nộp và xử phạt hành chính. Chatbot không nguồn tạo rủi ro kiểu khác: câu tiếng Việt trôi, kèm số Điều nghe quen, khiến người dùng tin hơn kết quả tìm từ khóa thô. Đồ án đặt yêu cầu tối thiểu: mọi căn cứ đưa ra UI phải mở được về một Điều trong kho. Yêu cầu này không biến hệ thống thành tư vấn pháp lý có trách nhiệm nghề nghiệp. Người dùng vẫn phải đối chiếu văn bản gốc và, khi tranh chấp, nhờ tổ chức hành nghề.

1.2.4 Ranh giới trợ lý AI và dịch vụ pháp lý

Hệ thống hỗ trợ tra cứu, nhắc việc và chỉ ra chỗ hợp đồng có thể lệch so với Điều đã truy hồi. Hệ thống không đại diện tố tụng, không soạn hồ sơ nộp cơ quan thuế, không cam kết “thắng kiện”. Câu trả lời mang tính hỗ trợ thông tin trên corpus đóng. Ranh giới này vừa là yêu cầu đạo đức nghề, vừa là phạm vi đồ án ngành: thời gian không đủ để xây phân hệ chữ ký số hay kênh nộp thật.

1.3 Bài toán tin học đặt ra từ hiện trạng

Từ hiện trạng trên, bài toán không phải “cài chatbot”. Ba lớp việc phải làm cùng lúc:

1. **Lớp dữ liệu.** Chuẩn hóa 38 văn bản thành Điều có siêu dữ liệu (số hiệu, chương, viện dẫn), nhập PostgreSQL, lập chỉ mục không dấu, nhúng vector.
2. **Lớp suy diễn hạn chế.** Pipeline RAG lai, hậu kiểm citation, lịch theo rule — suy diễn nằm ở chỗ ghép nguồn, không ở chỗ mô hình tự đặt ra hạn nộp.
3. **Lớp phần mềm.** Đa người dùng, JWT, cách ly tổ chức, SSE, Docker. Không có lớp này thì pipeline chỉ chạy được trên máy phát triển.

Báo cáo lần lượt trả lời ba lớp: Chương 2 cho kỹ thuật truy hồi và công nghệ; Chương 3 cho phân tích thiết kế; Chương 4 cho bằng chứng chạy được.

Sản phẩm là ứng dụng web đa người dùng: đăng nhập, lịch sử hội thoại, tra cứu văn bản, hỏi đáp có dẫn chiếu, soát xét hợp đồng và lịch tuân thủ. Pipeline hỏi đáp cắt corpus theo Điều, truy hồi lai BM25–vector, rồi sinh câu trên ngữ cảnh đã lọc. Mô hình mặc định là Gemini (lớp tương thích OpenAI). Đánh giá đồ án dựa trên kịch bản vận hành Docker và khả năng đối chiếu Điều gốc, không dựa trên bảng xếp hạng mô hình bên ngoài.

1.4 Lý do lựa chọn đề tài

DNNVV thường không duy trì bộ phận pháp chế riêng, trong khi nghĩa vụ định kỳ không giảm theo quy mô: kê khai thuế GTGT, tạm nộp và quyết toán TNDN, đóng BHXH, báo cáo lao động, nộp báo cáo tài chính năm. Các mốc này lặp lại, có hạn chót và gắn với chế tài khi chậm. Một công cụ hỗ trợ hỏi đáp có dẫn chiếu, tra cứu Điều gốc, soát hợp đồng và nhắc lịch nộp phù hợp hơn so với chatbot chỉ đưa lời khuyên chung.

Phạm vi được giới hạn ở thuế, lao động, bảo hiểm, kế toán, hợp đồng dân sự – thương mại và sở hữu trí tuệ mức cơ bản. Tổ tụng, hình sự và đất đai chuyên sâu không thuộc đề tài, nhằm bảo đảm hiện thực và kiểm chứng được trong thời gian đồ án ngành.

Về kỹ thuật, ba bài toán không giải được bằng cách cài một chatbot mẫu:

1. **Lệch từ vựng giữa người dùng và văn bản luật.** Truy hồi chỉ theo từ khóa dễ bỏ sót Điều đúng; truy hồi chỉ theo vector dễ nhầm Điều kề nhau. Cần kết hợp hai hướng rồi gộp hạng.
2. **Tìm kiếm tiếng Việt trên PostgreSQL.** Hệ quản trị này không có từ điển full-text tiếng Việt chuẩn. Người dùng thường gõ không dấu. Nếu tìm và hiển thị trên cùng bản đã bỏ dấu, đoạn trích mất dấu, khó đọc. Cần tách pha tìm (bản không dấu) và pha hiển thị (bản còn dấu).
3. **Hiện tượng bịa số Điều.** Nếu lấy đầu ra mô hình làm nguồn chính, hệ thống không khác chatbot không kiểm chứng. Cần hậu kiểm: số Điều trong câu trả lời phải thuộc tập vừa truy hồi.

1.5 Mục tiêu đề tài

1.5.1 Mục tiêu tổng quát

Xây dựng ứng dụng web đa người dùng, đa tổ chức, hỗ trợ DNNVV tra cứu và hỏi đáp pháp lý, trong đó mọi căn cứ pháp lý hiển thị đều truy được về Điều trong corpus đã chuẩn hóa.

1.5.2 Mục tiêu cụ thể

1. Xây dựng corpus DNNVV gồm 38 văn bản, 3.844 Điều, 452 chương; cắt theo Điều; lưu metadata và viện dẫn chéo.
2. Hiện thực pipeline hỏi đáp bảy bước: phân tích ý định, chuẩn bị truy vấn (HyDE), truy hồi lai, lọc ngữ cảnh, sinh câu, hậu kiểm trích dẫn; truyền kết quả theo SSE.
3. Xây dựng chức năng tra cứu theo cây Chương–Điều, tìm không dấu, hỗ trợ hỏi tiếp từ Điều đang đọc.
4. Xây dựng chức năng soát xét hợp đồng PDF/DOCX/TXT, xử lý nền, theo dõi trạng thái, xuất bảng rủi ro.
5. Xây dựng lịch tuân thủ 12 loại nghĩa vụ, sinh theo hồ sơ doanh nghiệp, kèm dẫn chiếu pháp lý.
6. Hiện thực xác thực JWT (access và refresh), bốn vai trò, cách ly dữ liệu theo người dùng và theo tổ chức.
7. Đóng gói bằng Docker Compose (ba dịch vụ).

1.6 Cách tiếp cận và phương pháp

1.6.1 Hỏi đáp dạng open-book

Hỏi đáp close-book dựa vào tri thức nén trong tham số mô hình, không chỉ được văn bản nguồn và dễ sai khi luật thay đổi. Đề án chọn open-book: mọi kết luận pháp lý phải dựa trên đoạn văn đã truy hồi từ corpus [3]. Khi không tìm được Điều liên quan, hệ thống thông báo thiếu căn cứ, không sinh đoạn khuyên chung giả danh quy định.

1.6.2 Truy hồi lai

Truy hồi dày (vector) bắt ngữ nghĩa; BM25 bắt mặt chữ và số hiệu. Reciprocal Rank Fusion gộp hai bảng xếp hạng mà không cần đưa điểm về cùng thang đo [6]. Cấu hình đề án: trọng số $\text{dense:BM25} = 3 : 1$, $k_{\text{RRF}} = 60$, lấy $k = 8$ Điều cho các bước sau.

1.6.3 Quy trình phát triển

Hệ thống được phát triển theo vòng ngắn: chuẩn hóa corpus, xây dựng CSDL và bảng tri thức, hiện thực xác thực và hội thoại, lần lượt hoàn thiện tra cứu, hợp đồng và lịch tuân thủ. Mỗi module hoàn thành API được gắn giao diện ngay, nhằm phát hiện sớm các lỗi chỉ xuất hiện khi vận hành (định tuyến số hiệu chứa dấu /, thứ tự bản ghi cùng mốc thời gian, tải tệp multipart).

Tìm kiếm toàn văn phụ thuộc tsvector, unaccent và pg_trgm trên PostgreSQL, không thay bằng SQLite.

1.7 Đối tượng và phạm vi

1.7.1 Đối tượng

- Đối tượng sử dụng: chủ DNNVV, kế toán, nhân sự — tra cứu và hỏi đáp bằng tiếng Việt thông thường.
- Đối tượng kỹ thuật: pipeline RAG lai trên corpus VBQPPL cắt theo Điều; ứng dụng web đa người dùng.

1.7.2 Phạm vi chức năng

Bốn nghiệp vụ: hỏi đáp có trích dẫn, tra cứu cây văn bản, soát xét hợp đồng, lịch tuân thủ. Đồ án không thực hiện OCR tệp ảnh, không nộp hồ sơ thuế hộ, không ký sổ và không đại diện tố tụng. Lịch tuân thủ là công cụ nhắc việc, không thay thế phần mềm kế toán.

1.7.3 Phạm vi dữ liệu và kỹ thuật

Corpus gồm 38 văn bản thuộc thuế, lao động, BHXH, kế toán, hợp đồng, sở hữu trí tuệ mức cơ bản và hỗ trợ DNNVV. Luật hình sự và văn bản địa phương không nằm trong kho. Mô hình ngôn ngữ và embedding được gọi qua mạng; đồ án không tự triển khai GPU. Soát xét hợp đồng dùng hàng đợi nội bộ BackgroundTasks của FastAPI, phù hợp quy mô đồ án, chưa phải hàng đợi phân tán.

1.8 Tổng quan các giải pháp hiện có

Bảng 1.1: Đối chiếu cổng VBQPPL, chatbot không nguồn và hướng đồ án.

Tiêu chí	Cổng VBQPPL	Chatbot không nguồn	không	Hướng đồ án
Cách hỏi	Số hiệu, từ khóa	Câu nói thường		Câu thường hoặc từ cây văn bản
Đối chiếu nguồn	Chính là văn bản luật	Không có dẫn chiếu mở được		Mỗi căn cứ mở đúng Điều
Bịa số Điều	Không xảy ra	Dễ gặp		Loại căn cứ ngoài tập truy hồi
Nghiệp vụ kế toán, nhân sự	Không	Không		Soát xét hợp đồng, lịch nộp
Độ rộng kho	Rất rộng	Không xác định nguồn	xác định	38 văn bản, phạm vi hẹp

Cổng VBQPPL của nhà nước vượt trội về độ tin cậy và độ phủ. Chatbot ngôn ngữ lớn vượt trội về cách hỏi tự nhiên. Đồ án không cạnh tranh độ phủ với cổng nhà nước. Hướng tiếp cận là

kết hợp câu hỏi ngôn ngữ tự nhiên với kho văn bản hẹp nhưng mở được từng Điều, đồng thời bổ sung hai nghiệp vụ mà cổng tra cứu không cung cấp: soát xét hợp đồng và nhắc mốc tuân thủ.

1.9 Ý nghĩa của đề tài

Đề án không đề xuất thuật toán truy hồi mới. Ý nghĩa thực tiễn nằm ở việc chuyển pipeline hỏi đáp thành hệ thống vận hành được cho DNNVV, cụ thể:

- Ứng dụng đa người dùng, đa tổ chức, có xác thực và cách ly dữ liệu, thay cho kịch bản hỏi đáp chạy một lần.
- Tra cứu tiếng Việt không dấu trên PostgreSQL trong khi đoạn trích hiển thị vẫn còn dấu; lịch tuân thủ sinh theo hồ sơ từng doanh nghiệp và dẫn được Điều có trong corpus.
- Hệ thống suy giảm có kiểm soát: khi thiếu khóa mô hình, tra cứu và lịch tuân thủ vẫn dùng được; hỏi đáp và soát xét hợp đồng báo chưa sẵn sàng.

1.10 Phương pháp nghiên cứu và công cụ

Đề án kết hợp nghiên cứu tài liệu và hiện thực hệ thống. Phần tài liệu gồm các công trình RAG, BM25, dense retrieval, RRF, HyDE và giáo trình công nghệ phần mềm [1]. Phần hiện thực tuân thủ vòng đời: phân tích yêu cầu, thiết kế kiến trúc, cài đặt, vận hành thử, đóng gói.

Công cụ chính: Visual Studio Code / Cursor để soạn mã; Git để lưu phiên bản; Docker Compose để dựng môi trường; XeLaTeX để biên soạn báo cáo. Dữ liệu pháp lý lấy từ văn bản đã công bố, không thu thập dữ liệu cá nhân ngoài tài khoản thử nghiệm.

1.11 Bố cục báo cáo

Báo cáo gồm phần đầu (lời cảm ơn, nhận xét, tóm tắt tiếng Việt, abstract tiếng Anh, mục lục, danh mục), năm chương, tài liệu tham khảo và phụ lục.

Chương 1 đặt bài toán DNNVV, mục tiêu, phạm vi và đối chiếu giải pháp hiện có. Chương 2 trình bày lý thuyết và công nghệ đề án thực sự dùng. Mỗi mục đi theo bốn câu (là gì, vì sao, cách dùng, khi nào) và lấy ví dụ pháp lý tiếng Việt, không chép định nghĩa bách khoa. Chương 3 phân tích yêu cầu, use case, thiết kế CSDL, API, danh mục 38 văn bản và 12 rule tuân thủ. Chương 4 mô tả môi trường thử, kịch bản Docker và tiêu chí định tính. Chương 5 kết luận, hạn chế, hướng phát triển.

Phụ lục hướng dẫn chạy hệ thống, thuật ngữ ngắn, quy trình nạp corpus và ràng buộc sử dụng.

Chương 2. CƠ SỞ LÝ THUYẾT

Chương này chỉ trình bày khái niệm *đồ án thực sự dùng*. Mỗi kỹ thuật và công nghệ đi theo bốn câu, để người đọc khỏi lẫn lộn trang giáo trình với phần đã cài:

- **Là gì** — nói ngắn, bằng việc đồ án cần làm, không chép định nghĩa bách khoa.
- **Vì sao dùng** — chỗ lệch từ vựng, chỗ bịa số Điều, hoặc chỗ tiếng Việt không dấu mà khái niệm đó gõ được.
- **Cách dùng** — file, tham số, nút pipeline.
- **Khi nào** — lúc nào khởi chạy; lúc nào bị tắt hoặc bỏ qua.

Mục so sánh ở cuối chương nêu phương án đã xem nhưng không chọn.

2.1 Xử lý ngôn ngữ tự nhiên trong bài toán hỏi đáp

Bài toán của đồ án nghe đơn giản: người dùng gõ một câu tiếng Việt, máy trả lời và chỉ ra Điều nào đang dùng. Khác hỏi đáp trên Wikipedia tiếng Anh, kho luật Việt Nam có khung cứng (Chương, Điều, Khoản), thuật ngữ được chuẩn hóa, và một số Điều sai có thể kéo theo tiền phạt. Vì vậy đồ án không coi đây là chatbot tán gẫu, mà là hỏi đáp trên kho đóng: câu trả lời phải bám đoạn văn đã tìm được [4, 3].

Hai hướng thường gặp:

- **Đóng sách (close-book)**. Mô hình trả lời từ những gì đã được nén vào trọng số lúc huấn luyện. Nhanh, nhưng không chỉ được trang luật nào. Khi nghị định đổi, câu trả lời cũ vẫn nghe trôi.
- **Mở sách (open-book)**. Máy tìm vài Điều liên quan, rồi mới đọc và viết. Đồ án theo hướng này. Nếu không tìm ra Điều, hệ thống nói thiếu căn cứ, không bịa đoạn khuyên chung.

Ví dụ dễ hình dung: câu “thử việc được bao lâu” và câu “sa thải nhân viên kinh doanh” dùng chữ đời thường. Văn bản gốc viết “thời gian thử việc” và “đơn phương chấm dứt hợp đồng lao động”. Máy phải vừa khớp mặt chữ (khi người dùng gõ đúng “Điều 25”), vừa bắt gần nghĩa (khi chữ hoàn toàn khác).

2.2 Các khái niệm nền gắn với tiếng Việt và kho luật

2.2.1 Tách từ và hai lớp token

Tiếng Việt viết cách âm tiết, không cách từ. “Lao động” là một từ, hai âm tiết. Nếu BM25 đếm “lao” và “động” riêng, độ hiếm của từ bị loãng và Điều đúng dễ bị Điều khác lấn. Đồ án tách từ

bằng `underthesea` khi dựng chỉ mục BM25. Kênh nhúng và mô hình sinh dùng cách tách của nhà cung cấp (subword). Hai lớp token khác nhau; đồ án chấp nhận và không ép một tokenizer cho cả hai.

- **Là gì.** Đơn vị chữ đưa vào BM25 (từ tiếng Việt) khác đơn vị chữ đưa vào mô hình sinh (subword).
- **Vì sao dùng.** Không tách từ thì BM25 yếu trên kho luật, vì nhiều cụm hai âm tiết.
- **Cách dùng.** Tách lúc dựng cache BM25. Embedding không dùng kết quả `underthesea`.
- **Khi nào.** Mỗi lần lập hoặc làm mới chỉ mục chữ. Không chạy trên trang tra cứu FTS.

2.2.2 Từ phổ biến và độ hiếm

Văn bản luật lập “Điều”, “khoản”, “quy định”. Người dùng vẫn hay gõ “Điều 25”. Đồ án không tự soạn danh sách từ dùng riêng cho pháp lý: BM25 đã hạ điểm những từ xuất hiện ở quá nhiều Điều (IDF). Giữ nguyên “Điều” giúp câu có số hiệu không bị mất tín hiệu.

2.2.3 Người tìm, máy đọc

Người dùng DNNVV không chọn trước Chương nào để máy đọc. Hệ thống phải tự tìm vài Điều, rồi mới viết câu [4]. Đồ án: kênh tìm là BM25 cộng Chroma; kênh đọc là mô hình sinh câu, không phải mô hình khoanh một đoạn kiểu đề thi đọc hiểu. Một câu thuế thường cần hơn một Điều.

Trang `/laws` chỉ tìm và hiện chữ, không gọi kênh đọc.

2.2.4 Bịa nội dung

Mô hình vẫn có thể viết điều không có trong ngữ cảnh, hoặc không có thật. Ở đây dạng nguy hiểm nhất là bịa số Điều hoặc bịa mức phạt. Đồ án chặn bằng ba lớp: bắt buộc có ngữ cảnh đã tìm; lời nhắc cấm bịa; sau khi viết xong chỉ giữ số Điều thuộc tập vừa tìm. Diễn giải sai trên Điều đúng thì ba lớp này không bắt — hạn chế Chương 5.

2.2.5 Cửa sổ ngữ cảnh và nhiệt độ

Mô hình nhìn các token trong một cửa sổ có hạn; chi phí tăng nhanh khi chuỗi dài [19]. Đó là lý do không nhét cả kho 3.844 Điều vào một lời nhắc, mà cắt còn 8 Điều sau lọc. Nhiệt độ chỉnh mức “mạo hiểm” khi chọn chữ tiếp theo. Đồ án để nhiệt độ 0 khi viết câu, để cùng một câu hỏi và cùng một tập Điều cho ra lời diễn giải ổn định hơn.

Đồ án không cài mạng Transformer: gọi API đã huấn luyện. Cổng sinh (LLM_MODEL) tách khỏi cổng nhúng (EMBEDDINGS_MODEL).

2.2.6 Độ chính xác, độ phủ và cách đánh giá

Trong truy hồi thông tin, độ chính xác hỏi “trong các Điều đưa ra, bao nhiêu Điều đúng”; độ phủ hỏi “trong các Điều đáng ra phải tìm, máy giữ được bao nhiêu”. Đề án thiết kế phễu 60 → 8 theo hướng độ phủ: bỏ sót hại hơn lấy thừa. Chương 4 không công bố F_2 , MRR hay ROUGE trên bộ nhãn nội bộ — chưa gắn nhãn đủ lớn. Việc đánh giá sản phẩm là chạy cụm Docker và đối chiếu Điều gốc bằng tay.

2.3 Xác thực, ủy quyền và ba lớp phần mềm

Hội thoại và hợp đồng không được nhìn chéo giữa hai công ty. Đối giao diện không được dùng công thức RRF. Đề án tách ba lớp: trình bày (React), nghiệp vụ (FastAPI, pipeline), dữ liệu (PostgreSQL, Chroma) [1]. Xác thực trả lời “ai đang gọi”; ủy quyền trả lời “được làm gì”. JWT đi kèm kiểm vai trò và cách ly theo `user_id / organization_id`. Khi đoán định danh hội thoại người khác, API trả 404 chứ không 403, để khởi lộ tài nguyên đang tồn tại. HTTPS chưa bật trong Compose nội bộ.

2.4 Cấu trúc văn bản quy phạm pháp luật Việt Nam

2.4.1 Luật, nghị định, thông tư không nằm trên một trang

Câu “công ty mười hai lao động khai GTGT theo tháng hay theo quý” không nằm gọn trong một Điều của Luật Quản lý thuế. Luật nói nghĩa vụ kê khai tồn tại. Nghị định nói ngưỡng và kỳ. Thông tư nói tờ khai. Nếu kho chỉ có luật, máy trả lời đúng nguyên tắc và sai thao tác — kiểu sai khó phát hiện vì nghe “có vẻ đúng luật”. Vì vậy đề án đưa cả ba cấp vào kho [10, 8, 9].

2.4.2 Điều là đơn vị cắt dữ liệu

Văn bản chia Phần – Chương – Mục – Điều – Khoản – Điểm. Đề án cắt theo **Điều** vì ba lý do thực dụng:

1. Người làm kế toán và nhân sự dẫn chiếu “Điều n luật X ”, không dẫn “đoạn 400 ký tự thứ 17”.
2. Một Điều thường đủ nghĩa để đứng trong lời nhắc mô hình. Cắt giữa câu làm mất chủ ngữ của khoản, dễ khiến mô hình đọc sai ai phải làm gì.
3. Độ dài trung bình khoảng hơn một nghìn ký tự: nhét được vài Điều vào ngữ cảnh mà chưa đầy cửa sổ.

Cắt cả bộ luật thành một vector thì vô dụng: máy không biết chỗ nào trả lời câu hỏi. Cắt theo số ký tự cố định thì hợp bài xử lý ngôn ngữ chung, nhưng phá cách dẫn chiếu của ngành luật.

2.4.3 Viện dẫn chéo

Kho gốc có trường extra: danh sách chỗ một Điều trở sang Điều khác. Đồ án không xây đồ thị tri thức đầy đủ. Viện dẫn được lưu và hiện thành liên kết trên trang đọc. Người đang xem Điều 44 Luật Quản lý thuế thấy đường sang Điều nói thời hạn nộp. Khi chạy thử, người không học luật dùng chỗ “nhảy” này nhiều hơn ô chat.

2.5 Truy hồi theo chữ: BM25

Khi người dùng gõ đúng cụm có trong văn bản — “Điều 25”, “thử việc”, “hóa đơn điện tử” — máy cần một cách chấm điểm dựa trên chữ, không dựa trên nghĩa. Đồ án dùng BM25 cho việc đó [5]. Ý tưởng: từ nào hiếm trong kho mà xuất hiện trong Điều thì được cộng nhiều; Điều quá dài bị trừ nhẹ để khỏi thắng chỉ vì dài.

- **Là gì.** Cách xếp hạng Điều theo mức khớp mặt chữ với câu hỏi, có tính độ hiếm của từ và độ dài văn bản [5].
- **Vì sao dùng.** Người dùng hay gõ số hiệu. Kênh vector để coi “Điều 24” và “Điều 25” là gần nhau vì chúng đứng cạnh trong cùng chương. BM25 phân biệt được hai số đó.
- **Cách dùng.** Chạy cùng lúc với Chroma trong nút `retrieve`. $k_1 = 1,2$, $b = 0,65$. Tách từ tiếng Việt bằng `underthesea` (backend ghim Python 3.10), kết quả tách được ghi cache.
- **Khi nào.** Mọi câu hỏi pháp lý đi truy hồi lai. Không chạy khi ý định là SKIP hoặc khi chưa có kho.

Công thức chuẩn được giữ nguyên. Q là câu hỏi, D là một Điều, $f(t, D)$ là số lần từ t xuất hiện trong Điều, $|D|$ là độ dài Điều, avgdl là độ dài trung bình trong kho, N là số Điều, $n(t)$ là số Điều chứa t :

$$\text{BM25}(Q, D) = \sum_{t \in Q} \text{IDF}(t) \cdot \frac{f(t, D) \cdot (k_1 + 1)}{f(t, D) + k_1 \cdot \left(1 - b + b \cdot \frac{|D|}{\text{avgdl}}\right)} \quad (2.1)$$

$$\text{IDF}(t) = \ln \left(\frac{N - n(t) + 0.5}{n(t) + 0.5} + 1 \right) \quad (2.2)$$

Hệ số b giảm nhẹ so với mặc định vì độ dài Điều trong kho dao động mạnh (điều định nghĩa ngắn, điều xử phạt dài). BM25 chịu thua khi từ vụng lệch hoàn toàn (“sa thải” / “đơn phương chấm dứt”) — lúc đó đồ án dựa vào kênh vector.

2.6 Truy hồi theo nghĩa: embedding và cosine

Kênh thứ hai biến mỗi Điều thành một dãy số (embedding). Hai dãy gần nhau theo góc (cosine) thì nội dung gần nghĩa, dù chữ khác nhau. Đó là lý do câu “sa thải” vẫn kéo được Điều nói đơn

phương chấm dứt.

- **Là gì.** Mỗi Điều (và mỗi câu hỏi) được đổi thành một vector; độ gần đo bằng cosine.
- **Vì sao dùng.** Bắt cặp gần nghĩa khi mặt chữ không trùng.
- **Cách dùng.** Mô hình gemini-embedding-001, cắt còn 1536 chiều. Nhúng theo Điều, lô 64 bản ghi, nghỉ dần khi gặp mã 429. Đổi số chiều thì xóa Chroma và nhúng lại.
- **Khi nào.** Lúc nạp hoặc lập lại chỉ mục (mọi Điều) và lúc hỏi (nhúng câu hoặc đoạn HyDE). Thiếu khóa nhúng: hỏi đáp trả 503, trang tra cứu vẫn dùng được.

$$\text{sim}(q, d) = \frac{\mathbf{v}_q \cdot \mathbf{v}_d}{\|\mathbf{v}_q\| \|\mathbf{v}_d\|} \quad (2.3)$$

Cosine chuẩn hóa độ dài vector, nên Điều dài không tự thắng Điều ngắn chỉ vì có nhiều số.

2.7 Gộp hai bảng xếp hạng: Reciprocal Rank Fusion

Điểm BM25 và điểm cosine không cùng một thước. Cộng thẳng hai cột số là tự bịa thang đo. Reciprocal Rank Fusion chỉ nhìn thứ hạng: Điều đứng hạng 1 được cộng nhiều, hạng 20 được cộng ít, không cần biết điểm thô là 12 hay 0,83 [6].

- **Là gì.** Cách gộp nhiều danh sách xếp hạng bằng nghịch đảo hạng [6].
- **Vì sao dùng.** Hai kênh trả về hai thứ tự khác nhau; cần một danh sách chung trước khi lọc và viết câu.
- **Cách dùng.** $k = 60$, trọng số nghĩa : chữ = 3 : 1, lấy 8 Điều đưa sang bước sau. Tỷ lệ 3:1 chọn sau khi thử tay: đa số câu DNNVV là câu nghĩa, nhưng BM25 vẫn kéo đúng khi người dùng gõ “Điều 35”.
- **Khi nào.** Trong nút retrieve, sau khi hai kênh trả danh sách. Nếu một kênh rỗng hoàn toàn thì dùng kênh còn lại.

$$\text{RRF}(d) = \sum_i \frac{w_i}{k + \text{rank}_i(d)} \quad (2.4)$$

w_i là trọng số kênh, $\text{rank}_i(d)$ là hạng của Điều d trên kênh i (hạng 1 là tốt nhất). Hằng số k làm giảm khoảng cách giữa hạng gần nhau, tránh một kênh lấn hết.

2.8 RAG, HyDE, lời nhắc mô hình và rủi ro bịa nguồn

2.8.1 RAG

Mô hình ngôn ngữ lớn viết tiếng Việt trôi, nhưng không “mở được Điều 44” trừ khi đoạn Điều được đưa vào lời nhắc. RAG buộc đúng việc đó: tìm trước, viết sau [3]. Khi luật đổi, người vận hành nhập văn bản mới, không bắt buộc huấn luyện lại mạng.

- **Là gì.** Quy trình tìm vài Điều liên quan, rồi mới viết câu trên đúng những Điều đó [3].
- **Vì sao dùng.** Không lấy trí nhớ trong trọng số làm nguồn luật: dễ bịa số Điều, khó cập nhật khi văn bản đổi.
- **Cách dùng.** Bấm nút LangGraph. Sau khi viết xong, hệ thống chỉ giữ số Điều thuộc tập vừa tìm. Nút lọc bằng mô hình giảm Điều lạc. Xếp hạng chéo tắt vì Gemini không có cổng rerank.
- **Khi nào.** Mỗi lượt hỏi đáp và mỗi điều khoản hợp đồng (cùng một hàm tìm). Không chạy khi thiếu chỉ mục vector (HTTP 503).

Rủi ro còn lại: mô hình diễn giải sai trên Điều đúng. Hàng rào số hiệu không bắt được lỗi đó — hạn chế nêu lại ở Chương 5.

2.8.2 HyDE

Câu hỏi của chủ doanh nghiệp thường ngắn và giọng đời thường. Văn luật dài và dùng chữ khác. HyDE nhờ mô hình viết một đoạn ngắn “như thể đã trả lời bằng giọng quy phạm”, rồi dùng đoạn đó để tìm vector [7]. Đoạn giả có thể sai nội dung; đồ án không hiện đoạn này cho người dùng như đáp án. Mục đích chỉ là kéo câu hỏi lại gần giọng kho.

- **Là gì.** Sinh một đoạn giọng quy phạm rồi nhúng đoạn đó để tìm vector [7].
- **Vì sao dùng.** Câu hỏi ngắn, lệch từ vựng so với văn bản gốc.
- **Cách dùng.** Mô hình viết khoảng 5–6 câu. Viết lại nhiều biến thể đang tắt để giữ độ trễ.
- **Khi nào.** Bật trên sản phẩm trước nút tìm. Tắt được trong `config.yaml`. Lỗi HyDE thì tìm bằng câu gốc.

2.8.3 Lời nhắc mô hình, không tinh chỉnh trọng số

Đồ án không có GPU để tinh chỉnh mô hình. Luật đổi thì nhập JSON rẻ hơn huấn luyện lại. Các nút chỉ được in đầu ra hẹp: ý định chỉ NEXT hoặc SKIP; lọc chỉ PASS hoặc DROP; câu trả lời chỉ được kết luận từ căn cứ đang có, mặc định chủ thể là DNNVV.

2.8.4 Chỉ số F_2

Khi thiết kế phiếu tìm (lấy 60 Điều lúc gộp, còn 8 Điều lúc viết), đồ án ưu tiên không bỏ sót Điều quyết định. Lấy thừa còn lọc được; bỏ sót thì không bước nào cứu. Chỉ số F_β với $\beta = 2$ diễn đạt đúng thiên hướng đó, nhưng Chương 4 *không* gắn một điểm F_2 làm kết quả sản phẩm: chưa có bộ câu gắn nhãn nội bộ đủ lớn.

$$F_\beta = (1 + \beta^2) \cdot \frac{P \cdot R}{\beta^2 P + R}, \quad \beta = 2 \quad (2.5)$$

P là độ chính xác (trong các Điều đưa ra, bao nhiêu Điều đúng), R là độ phủ (trong các Điều đáng ra phải tìm, máy giữ được bao nhiêu).

2.9 Tìm chữ tiếng Việt trên PostgreSQL

Trang tra cứu là đọc văn bản, không phải hỏi đáp. Người dùng hay gõ không dấu (“thue gtgt”). PostgreSQL không có từ điển full-text tiếng Việt sẵn. unaccent bỏ dấu lúc tìm; đoạn hiện trên trang vẫn là chữ gốc còn dấu [11].

- **Là gì.** Tìm chữ trên cột đã lập chỉ mục, không gọi mô hình [11].
- **Vì sao dùng.** Ô tìm trên /laws phải trả nhanh và đọc được khi mất khóa API.
- **Cách dùng.** Cột sinh sẵn: unaccent bọc hàm immutable_unaccent, cấu hình simple. Ba tầng: số Điều, ts_rank, pg_trgm (ngưỡng 0,3). Bôi đậm ở React trên chữ gốc, không dùng ts_headline (hàm này trả đoạn đã mất dấu).
- **Khi nào.** Mỗi lần gõ ô tìm. Không cần Chroma.

2.10 Công nghệ hiện thực và lý do chọn

Bảng 2.1 liệt kê ngắn xếp. Mục tiếp theo viết từng mục theo đúng bốn câu đã nêu.

2.10.1 HTTP, JSON và CORS

- **Là gì.** Cách trình duyệt nói chuyện với máy chủ: mỗi lần gọi độc lập, thân REST là JSON, khi dev hai cổng khác nhau thì trình duyệt hỏi CORS [17].
- **Vì sao dùng.** Toàn bộ nghiệp vụ đi qua HTTP. Body REST là JSON. Khi Vite (5173) gọi API (8023) khác origin, trình duyệt bắt buộc có CORS.
- **Cách dùng.** GET đọc, POST tạo, PATCH sửa, DELETE xóa. Mã dùng: 200, 201, 202 (soát xét nền), 400, 401, 403, 404 (kể cả xem chéo hội thoại), 422, 429, 503. CORS khóa origin cấu hình, không *. Docker cùng origin nên CORS gần như không chạy.

Bảng 2.1: Công nghệ gắn vào sản phẩm.

Thứ dùng		Nhóm	Việc trong đồ án
React	19,	Giao diện	SPA sau đăng nhập
TypeScript			
Vite + nginx		Đóng gói FE	Dev proxy / production proxy /api
Tailwind CSS 4		Trang trí	Không kéo bộ thành phần nặng
TanStack Query		Cache phía trình duyệt	Luật, hồi trạng thái soát xét
Zustand		Phiên	Người dùng và khôi phục token
FastAPI		Máy chủ	REST + SSE bất đồng bộ
SQLAlchemy 2 +		ORM / migration	Schema có phiên bản
Alembic			
PostgreSQL 17		CSDL	Nghiệp vụ + tìm chữ
ChromaDB		Vector	Cosine trên Điều
LangGraph		Điều phối	Bẫy nút pipeline
slowapi		Giới hạn tần suất	Bảo vệ hạn mức mô hình
Docker Compose		Cụm	Postgres, backend, frontend

- **Khi nào.** Mọi request API. OPTIONS chỉ khi trình duyệt preflight. HTTPS/TLS không bật trong Compose nội bộ — khi đưa ra Internet phải đặt trước nginx.

2.10.2 Python

- **Là gì.** Ngôn ngữ của toàn bộ tầng máy chủ: FastAPI, tách từ, LangGraph và SQLAlchemy chung một codebase.
- **Vì sao dùng.** underthesea (tách từ BM25) ghim Python 3.10; FastAPI, SQLAlchemy, LangGraph cùng một ngôn ngữ.
- **Cách dùng.** Quản lý gói bằng uv; type hint + Pydantic cho schema; async cho I/O tới Postgres, Chroma và Gemini.
- **Khi nào.** Toàn bộ tầng máy chủ. Không dùng cho frontend.

2.10.3 FastAPI

- **Là gì.** Cửa HTTP duy nhất của SPA: nhận REST và đẩy SSE, kiểm body bằng Pydantic, tự sinh mô tả OpenAPI [13].
- **Vì sao dùng.** Một request hỏi đáp chủ yếu chờ mạng: bất đồng bộ có lợi. SSE có StreamingResponse. Schema khớp TypeScript phía trình duyệt.
- **Cách dùng.** Router tách theo nghiệp vụ (auth, hội thoại, hỏi đáp, luật, tài liệu, lịch, quản trị). Pipeline RAG nằm tầng dịch vụ, không nằm trong router.

- **Khi nào.** Mọi HTTP nghiệp vụ. Đường /docs dùng khi trình bày API, không phải luồng người dùng DNNVV.

2.10.4 Pydantic

- **Là gì.** Lớp biên giới API: body sai thì 422, không vào nghiệp vụ; schema khớp TypeScript phía trình duyệt.
- **Vì sao dùng.** Lỗi 422 có cấu trúc khi client gửi sai trường; OpenAPI khớp schema thật.
- **Cách dùng.** Body đăng ký, đăng nhập, tin nhắn, kết quả soát xét, bộ câu chạy hàng loạt.
- **Khi nào.** Mỗi lần FastAPI nhận hoặc trả JSON có khai báo mô hình.

2.10.5 SQLAlchemy 2.0 và Alembic

- **Là gì.** Cách đọc ghi PostgreSQL bằng lớp Python, kèm nhật ký đổi cột để máy khác lên cùng schema.
- **Vì sao dùng.** CRUD hội thoại, tài liệu, lịch không viết SQL lặt. Không sửa schema tay trên máy rồi quên ghi.
- **Cách dùng.** Bản bắt đồng bộ + asyncpg. User ứng dụng không CREATE EXTENSION; extension nằm init.sql.
- **Khi nào.** Mọi đọc/ghi PostgreSQL nghiệp vụ. Vector không đi ORM — nằm Chroma.

2.10.6 PostgreSQL

- **Là gì.** Nơi giữ user, hội thoại, Điều chữ, lịch và ràng buộc (mã số thuê duy nhất, task không trùng kỳ).
- **Vì sao dùng.** Cần ràng buộc tổ chức–user–hội thoại, FTS tiếng Việt tự dựng, và lịch tuân thủ. Chroma không làm việc này.
- **Cách dùng.** Bản 17. Cổng host 23432. Corpus chữ và nghiệp vụ cùng một cụm; cột generated tsvector sau unaccent.
- **Khi nào.** Luôn chạy khi hệ thống sống. Tra cứu FTS không cần Gemini.

2.10.7 ChromaDB

- **Là gì.** Nơi giữ vector từng Điều và tìm k Điều gần câu hỏi theo cosine, chạy trong cùng tiến trình backend.
- **Vì sao dùng.** Nhẹ, chạy kèm tiến trình backend, đủ 3.844 Điều; không thêm cụm Pinecone.

- **Cách dùng.** Volume `chroma_db`. Manifest (mô hình, chiều, số bản ghi) so khớp lúc khởi động. Đổi chiều thì xóa và nhúng lại.
- **Khi nào.** Nạp/reindex và nút `retrieve`. Thiếu chỉ mục: hỏi đáp 503, FTS vẫn dùng.

2.10.8 LangGraph

- **Là gì.** Cách nối bảy bước hỏi đáp thành một đồ thị có nhánh thoát sớm và có sự kiện SSE từng nút.
- **Vì sao dùng.** Cần thoát sớm khi câu không phải pháp luật, và đẩy SSE theo từng nút. Không dùng agent tự chọn công cụ tự do: thứ tự truy hồi rồi sinh là cố định.
- **Cách dùng.** Bảy nút; bộ nhớ ngăn nạp từ PostgreSQL, không dùng InMemorySaver.
- **Khi nào.** Mỗi lượt hỏi đáp. Soát xét hợp đồng tái sử dụng hàm truy hồi, không đi đủ bảy nút hội thoại.

2.10.9 Uvicorn

- **Là gì.** Tiến trình chạy FastAPI trong Docker: một worker, cổng 8023, đủ để giữ SSE và không đưa file Chroma.
- **Vì sao dùng.** SSE cần vòng sự kiện bất đồng bộ; Gunicorn đồng bộ thuần không hợp.
- **Cách dùng.** Docker: một tiến trình, không reload, cổng 8023. Một worker để hai process không đưa file Chroma.
- **Khi nào.** Luôn khi backend sống. Reload chỉ trên máy phát triển ngoài Compose.

2.10.10 Mô hình ngôn ngữ và Gemini

- **Là gì.** Dịch vụ viết câu và nhúng chữ qua API (Gemini), không phải kho luật và không được tinh chỉnh trọng số trong đồ án.
- **Vì sao dùng.** Không có GPU để tự host. Lớp tương thích OpenAI cho phép đổi `base_url` sang vLLM sau này mà không viết lại nút.
- **Cách dùng.** `gemini-2.5-flash`, nhiệt độ 0, tối đa 4096 token. Nhúng tách `gemini-embedding-001`. Flash thay Pro: đủ câu ngắn trên $k = 8$ Điều, rẻ hơn trên HyDE + filter + generate.
- **Khi nào.** Intent, HyDE, filter, generate, và nhúng. Thiếu khóa: health 200, chat 503. Không fine-tune.

2.10.11 Underthesea

- **Là gì.** Công cụ tách từ tiếng Việt cho BM25, để “lao động” không bị đếm thành hai mảnh.
- **Vì sao dùng.** BM25 cần đơn vị từ. “Lao động” tách thành “lao” / “động” làm nhiều IDF.
- **Cách dùng.** Nạp được thì dùng; không thì fallback regex. Cache theo dấu vân tay tokenizer.
- **Khi nào.** Lúc dựng chỉ mục BM25 (khởi động hoặc cache lệch). Không dùng cho embedding.

2.10.12 React

- **Là gì.** Thư viện vẽ toàn bộ màn hình sau đăng nhập; đổi trang không tải lại cả site [14].
- **Vì sao dùng.** Đề bài yêu cầu React. Ứng dụng sau đăng nhập, không cần SSR/SEO.
- **Cách dùng.** React 19. Tách ChatPanel, CitationList, AgentTrace. Không dùng Next.js.
- **Khi nào.** Mọi màn hình sau login. Trang login/register cũng là component React.

2.10.13 TypeScript

- **Là gì.** JavaScript có kiểu: lệch tên trường API (token, citation) bị bắt lúc build.
- **Vì sao dùng.** Lệch access_token / accessToken bắt lúc biên dịch.
- **Cách dùng.** Khai báo user, hội thoại, Điều, finding, sự kiện SSE. Khớp OpenAPI FastAPI.
- **Khi nào.** Toàn bộ frontend.

2.10.14 Vite

- **Là gì.** Công cụ dựng SPA: lúc lập trình có proxy /api, lúc đóng gói ra tệp tĩnh cho nginx.
- **Vì sao dùng.** Khởi động nhanh; proxy /api giống nginx production.
- **Cách dùng.** VITE_API_BASE_URL trống trong Docker: cùng origin.
- **Khi nào.** Dev trên máy; image frontend chỉ chứa bản đã build.

2.10.15 Tailwind CSS

- **Là gì.** Cách trang trí giao diện bằng class nhỏ trong JSX, không kéo bộ nút thương mại.
- **Vì sao dùng.** Không kéo bộ component thương mại; gói nhỏ, nền tối đủ đọc Điều dài.
- **Cách dùng.** Bản 4. Nút, ô nhập, thẻ tự viết.
- **Khi nào.** Mọi trang giao diện.

2.10.16 TanStack Query

- **Là gì.** Bộ nhớ tạm phía trình duyệt cho danh mục luật và trạng thái soát xét, tách khỏi kho phiên đăng nhập.
- **Vì sao dùng.** Danh mục luật, cây Điều, poll soát xét — không nhét vào store phiên.
- **Cách dùng.** Poll 2,5 giây khi soát xét. Tắt `refetchOnWindowFocus`.
- **Khi nào.** Mọi GET danh sách/chi tiết. Không dùng cho luồng SSE (Fetch).

2.10.17 Zustand

- **Là gì.** Kho nhỏ phía trình duyệt: chỉ user, token và hàm khôi phục phiên.
- **Vì sao dùng.** Chỉ cần user + token + restore; Redux thừa boilerplate.
- **Cách dùng.** Đọc token `localStorage` khi tải trang. Không chứa danh sách luật.
- **Khi nào.** Khởi động SPA và mỗi lần login/logout.

2.10.18 React Router

- **Là gì.** Cách đổi URL mà không tải lại trang; số hiệu có dấu / cần bắt phần còn lại của đường dẫn.
- **Vì sao dùng.** Số hiệu văn bản chứa / cần splat `*`.
- **Cách dùng.** Route `/login`, `/chat/:id`, `/laws/:lawId/*`, `/contracts`, `/compliance`, `/admin/*`, `/admin/batch-eval`. Admin `lazy()`. `RequireAuth` chờ restore xong.
- **Khi nào.** Mỗi lần người dùng đổi trang.

2.10.19 Axios

- **Là gì.** Client REST: gán Bearer, làm mới token một lần khi 401, gỡ JSON khi gửi file.
- **Vì sao dùng.** Cần interceptor chung: Bearer, làm mới token một lần, hàng đợi 401.
- **Cách dùng.** Xóa `Content-Type` khi `FormData` để trình duyệt đặt multipart.
- **Khi nào.** Mọi REST. Không dùng cho SSE — hỏi đáp dùng Fetch.

2.10.20 nginx

- **Là gì.** Cổng duy nhất người dùng mở: phát tệp Vite và chuyển /api vào backend, tắt đệm cho SSE.
- **Vì sao dùng.** Trình duyệt chỉ mở một cổng; /api vào backend.
- **Cách dùng.** Image frontend phục vụ bản Vite. Host 5173.
- **Khi nào.** Môi trường Docker / production. Dev dùng proxy Vite thay nginx.

2.10.21 Docker và Docker Compose

- **Là gì.** Cách dựng cùng một cụm Postgres-API-nginx trên máy khác nhau, không lệch phiên bản Python hay CSDL [15].
- **Vì sao dùng.** Cùng Python 3.10, Postgres 17, nginx trên máy khác nhau. Không lệch phiên bản khi chấm đồ án.
- **Cách dùng.** Ba dịch vụ: postgres (healthcheck pg_isready), backend (đợi healthy, /health), frontend. Compose chỉ đọc .env gốc, không đọc backend/.env.
- **Khi nào.** Lúc dựng và vận hành cụm. Máy dev có thể chạy Vite + Postgres ngoài Compose.

2.10.22 JWT

- **Là gì.** Hai loại token ký sẵn (access ngắn, refresh dài) để API biết ai đang gọi mà không giữ phiên trên RAM [12].
- **Vì sao dùng.** Không thêm Redis. Access ngắn hạn giảm cửa sổ lộ token.
- **Cách dùng.** HS256, JWT_SECRET_KEY. Claim sub, type (access/refresh), iat, exp, jti. Access 15 phút, refresh 7 ngày. Gọi API bằng refresh bị từ chối.
- **Khi nào.** Mọi request sau login. Token ở localStorage — còn rủi ro XSS; hướng phát triển là cookie HttpOnly.

2.10.23 Argon2

- **Là gì.** Cách lưu mật khẩu: băm argon2id, cố ý chậm, không lưu chữ gốc [20].
- **Vì sao dùng.** SHA-256 quá nhanh, dễ dò GPU. Cần hàm cố ý chậm và tốn bộ nhớ.
- **Cách dùng.** Qua passlib. Không lưu plaintext. Hash hỏng không sập login — verify trả sai.
- **Khi nào.** Đăng ký và mỗi lần đăng nhập. Không dùng cho JWT.

2.10.24 Server-Sent Events

- **Là gì.** Kênh máy chủ đẩy tiến độ và từng chữ xuống trình duyệt trong một kết nối HTTP, không mở WebSocket.
- **Vì sao dùng.** Cần tiến độ từng nút và từng token. WebSocket thừa chiều client; polling chat giật chữ.
- **Cách dùng.** Ba loại: status, token, result. Heartbeat tránh proxy cắt. Client dùng Fetch (POST + Bearer) vì EventSource không gán header.
- **Khi nào.** Mỗi lượt hỏi đáp. Soát xét hợp đồng dùng polling 2,5 giây, không SSE.

2.10.25 slowapi

- **Là gì.** Chặn một tài khoản gọi chat hoặc tải file quá dày, để khỏi vét hạn mức mô hình.
- **Vì sao dùng.** Một tài khoản không được vét hạn mức Gemini.
- **Cách dùng.** Chat 20 lần/phút, tải tệp 30 lần/giờ, theo user (hoặc IP nếu chưa login).
- **Khi nào.** Các endpoint tốn mô hình hoặc tốn đĩa. Tra cứu FTS không siết cùng mức chat.

2.10.26 Kiến trúc client-server và SPA

- **Là gì.** Tách giao diện và máy chủ: trình duyệt tải một bộ JS, mọi nghiệp vụ đi /api/v1.
- **Vì sao dùng.** Tách giao diện và máy chủ; trang sau login không cần HTML sẵn cho SEO.
- **Cách dùng.** Tiền tố /api/v1. Ba môi trường: Vite + Postgres local, Compose ba container, tương lai tách domain.
- **Khi nào.** Toàn bộ sản phẩm. Không dùng SSR.

2.11 Chi tiết từng nút pipeline LangGraph

Mỗi nút đọc vài trường của LegalAssistantState, ghi vài trường, phát SSE status. Prompt gốc không trích toàn văn (dài, dễ lệch khi sửa mã).

2.11.1 Nút analyze_intent

- **Là gì.** Phân loại câu vào ô chat: cần tra luật hay trả lời ngắn.
- **Vì sao dùng.** Tránh nhúng và gọi retrieve cho “xin chào”, “cảm ơn”, câu ngoài miền.
- **Cách dùng.** LLM chỉ được in NEXT hoặc SKIP. Nhấn khác hoặc lỗi mạng → ép NEXT (thà tra thừa). Bộ nhớ ngắn đã gán từ PostgreSQL trước khi gọi.

- **Khi nào.** Mọi lượt chat. Trang chạy hàng loạt luôn NEXT để không câu nào bị SKIP.

2.11.2 Nút `prepare_retrieval_query`

- **Là gì.** Tạo văn bản dùng để tìm: câu gốc, câu viết lại, hoặc đoạn HyDE.
- **Vì sao dùng.** Câu người dùng ngắn, giọng đòi thường; văn luật khác từ vựng.
- **Cách dùng.** Sản phẩm: HyDE bật, rewrite tắt. Ưu tiên đoạn HyDE, rồi câu gốc. Lỗi một nhánh thì dùng nhánh còn lại. Intent SKIP thì không gọi LLM.
- **Khi nào.** Sau intent NEXT. Tắt/bật bằng `config.yaml`.

2.11.3 Nút `retrieve`

- **Là gì.** Gọi `search_legal_articles`: Chroma + BM25, gộp RRF, lấy $k = 8$.
- **Vì sao dùng.** Cần tập Điều cho filter và generate. Tìm đồng bộ chạy luồng phụ để heartbeat SSE không bị nghẽn.
- **Cách dùng.** Lai, trọng số 3:1, $k_{\text{RRF}} = 60$. `search_spaces` từ client chỉ giao registry đã nạp.
- **Khi nào.** Intent NEXT và registry sẵn. SKIP: danh sách rỗng. Ngoại lệ: retrieved rỗng, generate nói thiếu căn cứ — không bịa từ tham số mô hình.

2.11.4 Nút `rerank`

- **Là gì.** Nút giữ chỗ cho xếp hạng chéo cặp (câu hỏi, Điều).
- **Vì sao dùng.** Cross-encoder chính xác hơn cosine nhưng đắt; Gemini không có cổng rerank.
- **Cách dùng.** Trên sản phẩm **tắt**: sao chép nguyên danh sách retrieve. Lỗi cũng fallback, không nuốt ứng viên.
- **Khi nào.** Chỉ khi tự host endpoint rerank và bật trong cấu hình.

2.11.5 Nút `llm_filter`

- **Là gì.** Hỏi PASS/DROP từng Điều: “có giúp trả lời câu hỏi không?”
- **Vì sao dùng.** Thay rerank: giảm Điều lạc trước khi sinh câu.
- **Cách dùng.** Gọi song song, semaphore `max_concurrency`. Item lỗi mạng được giữ (PASS). Nếu PASS ít hơn `min_keep` thì lấy thêm từ danh sách gốc.
- **Khi nào.** Bật trên sản phẩm sau retrieve. Không chạy khi retrieved rỗng.

2.11.6 Nút `generate_answer`

- **Là gì.** Sinh câu tiếng Việt trên prompt hệ thống + Điều đã lọc + vài lượt gần nhất.
- **Vì sao dùng.** Đây là chỗ người dùng đọc kết luận; nhiệt độ 0 để diễn giải ổn định.
- **Cách dùng.** Tối đa 4096 token; stream SSE token. SKIP: không nhét Điều, không citation. Tập rỗng: nhắc thừa nhận thiếu nguồn. Hậu kiểm citation nằm kề API, không trong nút.
- **Khi nào.** Sau filter (hoặc sau retrieve nếu filter tắt).

2.11.7 Nút `format_submission`

- **Là gì.** Gom doc_ref, article_ref, viện dẫn chéo đã lưu ở extra.
- **Vì sao dùng.** UI cần danh sách căn cứ bấm được; lượt sau cần tin nhắn AI trong state.
- **Cách dùng.** Không gọi LLM. Không truy hồi thêm.
- **Khi nào.** Luôn chạy cuối pipeline, kể cả nhánh SKIP (danh sách căn cứ rỗng).

Bảng 2.2 là cấu hình sản phẩm tại thời điểm báo cáo.

Bảng 2.2: Bảy nút LangGraph trên cấu hình sản phẩm.

Nút	LLM?	Khi nào / ghi chú
<code>analyze_intent</code>	Có	SKIP/NEXT; lỗi thì NEXT.
<code>prepare_retrieval_query</code>	Có (HyDE)	Rewrite tắt; HyDE bật.
<code>retrieve</code>	Nhúng câu hỏi	Hybrid RRF 3:1, $k = 8$.
<code>rerank</code>	Không (tắt)	Giữ nguyên retrieved.
<code>llm_filter</code>	Có	PASS/DROP; min_keep.
<code>generate_answer</code>	Có	Nhiệt độ 0; stream token.
<code>format_submission</code>	Không	Chuẩn hoá ref, ghi memory.

2.12 So sánh phương án và lý do không chọn

Các mục trước đã trả lời bốn câu cho cái đề án *dùng*. Mục này nêu cái đã xem rồi bỏ: phương án là gì, vì sao không chọn, và khi nào sẽ xem lại.

2.12.1 REST và GraphQL

GraphQL cho client tự chọn trường, giảm overfetch. Đề án schema ổn định, trang nào cũng cần gần đủ trường Điều/citation; lợi GraphQL nhỏ, thêm endpoint `/graphql` và cache phức tạp hơn. REST + OpenAPI khớp FastAPI có sẵn.

2.12.2 SSE, WebSocket và polling

WebSocket hai chiều, hợp chat typing đối phương. Đề án chỉ server đẩy tiến độ và token: SSE đủ, đi qua nginx đơn giản hơn WS. Polling cho soát xét (2,5 giây) vì việc nền không cần stream token; polling chat thì lãng phí và giật chữ.

2.12.3 JWT và phiên Redis

Redis session dễ thu hồi. Đề án không thêm dịch vụ Redis: JWT stateless, access ngắn. Thu hồi sớm (khóa user) kiểm is_active lúc mỗi request — đủ đề án, chưa denylist jti.

2.12.4 PostgreSQL và MongoDB

Mongo linh hoạt JSON. Pháp lý cần ràng buộc, FTS, giao dịch đăng ký+tổ chức. Postgres JSONB vẫn chứa extra và citation. Không cần hai CSDL.

2.12.5 Chroma, FAISS và Pinecone

FAISS mạnh khi tự host CPU/GPU, thiếu metadata store sẵn. Pinecone SaaS thêm tài khoản và mạng. Chroma nhúng process, volume Docker, đủ 3.844 vector.

2.12.6 Next.js và Vite SPA

Next.js SSR/SEO. Mọi trang nghiệp vụ sau login: SEO không phải yêu cầu. Vite SPA đơn giản, khớp đề bài React. Không dùng Next chỉ để “thêm cho có”.

2.12.7 Redux Toolkit và Zustand

Redux hợp state lớn, middleware nhiều. Đề án một store phiên: Zustand ít boilerplate.

2.12.8 Celery/Redis và BackgroundTasks

Hàng đợi thật chịu chết worker, retry, nhiều máy. Soát xét đề án một container: BackgroundTasks. Khi tệp dài và cần chắc chắn, chuyển Celery — hướng Chương 5.

2.12.9 Cookie HttpOnly và localStorage

Cookie an toàn XSS hơn, thêm CSRF. localStorage nhanh cho SPA Axios. Đề án chọn cái sau, ghi nợ cái trước.

2.12.10 Fine-tune và prompt

Fine-tune tốn GPU, luật đổi phải train lại. Prompt + RAG cập nhật bằng nhập JSON. Đề án không fine-tune.

2.13 Bảng tổng kết: vai trò – thách thức – hướng xử lý

Ba bảng dưới đây chốt việc mỗi khối phải làm gì trong *đồ án này*, chứ không lặp lại định nghĩa ở trên.

Bảng 2.3: Tổng kết khối truy hồi lại.

Hạng mục	Nội dung
Vai trò	Đưa đúng tập Điều liên quan vào lời nhắc trước khi viết câu.
Thách thức	Lệch từ vựng người–luật; số hiệu dễ bị kênh vector làm tròn; hạn mức nhúng.
Hướng xử lý	BM25 + Chroma + RRF 3:1; HyDE; nhúng lô 64 kèm nghỉ dần; cache BM25.

Bảng 2.4: Tổng kết khối sinh câu và hậu kiểm trích dẫn.

Hạng mục	Nội dung
Vai trò	Viết câu tiếng Việt có thể đối chiếu với Điều gốc.
Thách thức	Mô hình bịa số Điều; diễn giải sai trên Điều đúng; đột hạn mức.
Hướng xử lý	Lời nhắc siết căn cứ; lọc PASS/DROP; gỡ citation ngoài tập tìm; nhiệt độ 0; thiếu khóa thì 503, không sập cụm.

Bảng 2.5: Tổng kết khối nghiệp vụ DNNVV (hợp đồng + lịch).

Hạng mục	Nội dung
Vai trò	Biến hỏi đáp thành việc kế toán và nhân sự làm hàng tuần.
Thách thức	PDF ảnh; hạn chốt luật đổi; cách ly dữ liệu nhiều công ty.
Hướng xử lý	Cắt điều khoản rồi dùng lại hàm tìm; 12 rule có <code>legal_refs</code> ; cách ly theo tổ chức và người dùng; từ chối PDF không có lớp chữ lúc tải.

2.14 Công trình liên quan

Đồ án dựa trên RAG và truy hồi dày [3, 4]. BM25 và RRF là công cụ xếp hạng đã được công bố [5, 6]. HyDE được dùng để đưa câu hỏi gần giọng văn bản luật hơn trước khi nhúng [7]. Công trình chatbot tuyển sinh [2] cho thấy RAG vận hành được trên dữ liệu hẹp tiếng Việt.

Điểm khác của đồ án: kho có cấu trúc Điều–Chương; tìm không dấu nhưng hiện còn dấu; lịch tuân thủ gần Điều trong kho; bốn nghiệp vụ trên một ứng dụng web đa người dùng. Phần hiện thực ở Chương 3; phần chạy thử ở Chương 4.

Chương 3. PHÂN TÍCH, THIẾT KẾ VÀ HIỆN THỰC

Chương này gắn yêu cầu vào hiện thực. Mỗi khối thiết kế (cắt Điều, bảy nút, FTS, JWT, lịch, Docker) được đọc theo cùng bốn câu như Chương 2: khối đó là gì, vì sao chọn, gắn vào hệ thống thế nào, và chạy khi nào.

3.1 Tổng quan chức năng từ góc nhìn người dùng

Người dùng tạo tài khoản, khai tên công ty, mã số thuế, số lao động, kỳ kê khai GTGT. Bốn việc chính sau đó:

Hỏi. Gõ câu hỏi, thấy từng bước (đang tìm, đang lọc, đang viết), thấy từng chữ hiện ra, thấy danh sách Điều bấm được. Hội thoại cũ nằm cột trái, sống sót khi restart máy chủ.

Đọc luật. Lọc theo lĩnh vực, vào một văn bản, đi từ Chương xuống Điều, bấm viện dẫn chéo, hoặc bấm “Hỏi AI về Điều này” để câu hỏi được điền sẵn kèm số hiệu.

Đưa hợp đồng. Kéo file PDF/DOCX/TXT, tối đa 10 MB. Máy trích chữ ngay (lỗi file lộ lúc tải, không đợi). Bấm soát xét thì chạy nền; trang hỏi trạng thái mỗi vài giây. Kết quả là bảng: điều khoản, mức cao/trung bình/thấp, đề xuất sửa, căn cứ.

Nhìn lịch. Các mốc trong năm, nhóm theo tháng, đánh dấu xong hoặc bỏ qua. Sinh lại khi đổi số lao động hay đổi kỳ GTGT.

Người quản trị (tài khoản đăng ký đầu tiên khi bảng user còn trống được gắn vai trò này) vào được kho văn bản, danh sách user, và trang chạy hàng loạt câu hỏi để kiểm tra pipeline.

3.2 Yêu cầu

3.2.1 Chức năng

3.2.2 Phi chức năng

Thời gian: trang tra cứu phải trả trong khoảng dưới một giây trên máy phát triển dùng cho đồ án; hỏi đáp chấp nhận hàng chục giây nếu hiện tiến độ. Bảo mật: mật khẩu argon2, CORS không phải *, giới hạn số lần gọi chat. Sẵn sàng: thiếu embedding thì phần còn lại vẫn sống.

3.3 Kiến trúc ba lớp

Lúc cài bằng Docker, trình duyệt chỉ nói chuyện với cổng frontend. Nginx vừa đưa file tĩnh vừa chuyển /api sang backend, nên không có chuyện CORS trên môi trường đó. Khi phát triển, Vite thực hiện chuyển tiếp tương tự trên cổng 5173.

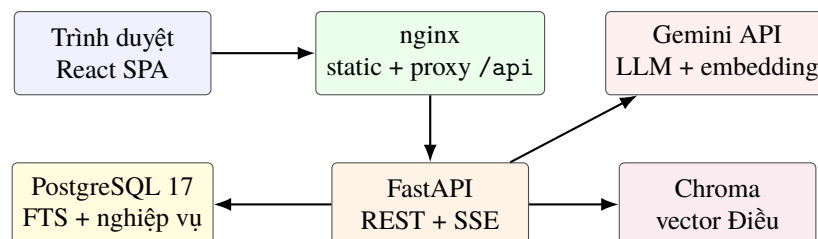
Logic RAG không nằm trong file router. Router chỉ nhận request, ghi hội thoại, gọi agent. Cùng một hàm truy hồi được soát xét hợp đồng gọi lại, tránh nhân bản prompt giữa hai module.

Bảng 3.1: Yêu cầu chức năng của hệ thống.

Mã	Việc	Ghi chú khi cài
YC-01	Đăng ký, đăng nhập, làm mới token, đổi mật khẩu	Access 15 phút, refresh 7 ngày, hai loại token không dùng lẫn
YC-02	Hỏi đáp có trích dẫn và stream	Ghi câu hỏi vào CSDL trước khi gọi pipeline
YC-03	Lịch sử hội thoại	Đổi tên, lưu trữ, xóa; không nhìn hội thoại người khác (trả 404)
YC-04	Danh mục và cây văn bản	Số hiệu có dấu / nên định tuyến phải kiểu path
YC-05	Tìm toàn văn không dấu	Ba tầng: số Điều, ts_rank, trigram
YC-06	Tải và soát xét hợp đồng	Trích chữ lúc tải; chấm điểm lúc soát xét
YC-07	Lịch tuân thủ	Idempotent: gọi sinh lại không xóa việc đã đánh dấu xong
YC-08	Quản trị corpus và user	Nhập JSON, reindex, khóa tài khoản
YC-09	Chạy hàng loạt câu hỏi	Chỉ admin; JSON danh sách câu, không dùng hội thoại

Bảng 3.2: Ngưỡng phi chức năng của hệ thống.

Nhóm	Ngưỡng	Cách kiểm
Độ trễ tra cứu	< 1 giây trên máy thử nghiệm với FTS	Gọi /laws/search
Độ trễ hỏi đáp	Chấp nhận hàng chục giây nếu có SSE tiến độ	Quan sát UI
Bảo mật mật khẩu	argon2id, tối thiểu 8 ký tự	Đăng ký trên UI
Cách ly dữ liệu	Hội thoại theo user; hợp đồng/lich theo org	404 khi xem chéo
Sẵn sàng suy giảm	Thiếu embedding: health 200, chat 503	Chạy Docker thiếu khóa
Giới hạn tần suất	Chat 20/phút; upload 30/giờ	slowapi

**Hình 3.1:** Ba lớp triển khai: trình duyệt → nginx → FastAPI; PostgreSQL, Chroma và Gemini nằm phía sau API.

BM25 không để Postgres. Khởi động backend: đọc Điều từ Postgres, nếu file cache khớp dấu vân tay tokenizer thì nạp cache, không thì tách từ lại và ghi cache. Vector: nếu manifest (model, số chiều, số record) lệch thì nhúng lại, có chia lô và nghỉ dần khi bị 429.

3.4 Use case

Use case mô tả tương tác giữa tác nhân và hệ thống để đạt một mục tiêu [1]. Mỗi use case có tiền điều kiện, luồng chính, nhánh và lỗi. Đồ án dùng use case làm cầu giữa yêu cầu ngôn ngữ tự nhiên và API.

Bảng 3.3: Tác nhân.

Tác nhân	Người nào	Được làm
Chủ doanh nghiệp	Người mở tài khoản công ty	Cả bốn nghiệp vụ trong công ty mình
Kế toán, Nhân sự	Người được mời vào cùng công ty	Cùng bốn nghiệp vụ; không vào trang quản trị
Quản trị	Tài khoản đầu, hoặc được đổi role	Thêm quyền corpus, user, chạy hàng loạt
Khách	Chưa đăng nhập	Chỉ đăng ký / đăng nhập

Bảng 3.4: Danh sách use case.

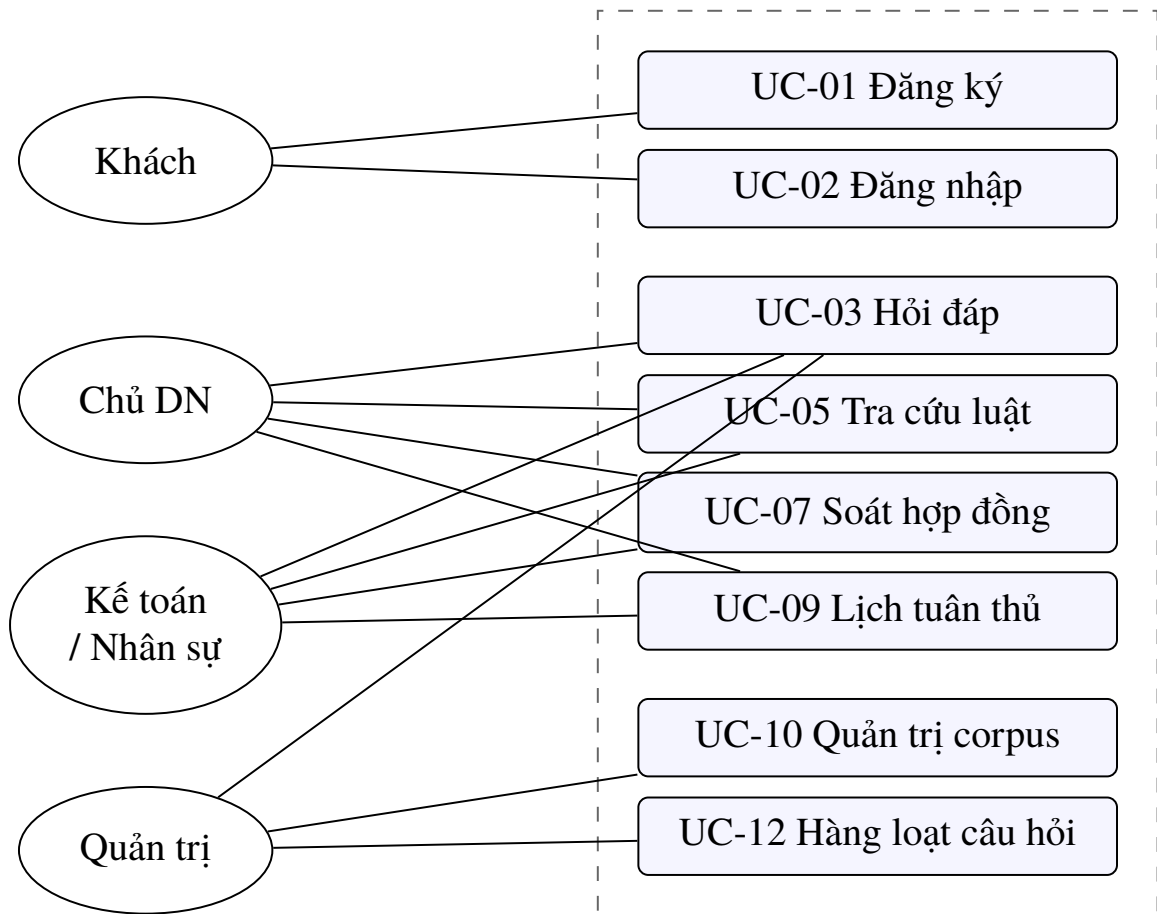
Mã	Tên	Ai
UC-01	Tạo tài khoản kèm hồ sơ công ty	Khách
UC-02	Đăng nhập, làm mới phiên	Mọi vai trò đã có tài khoản
UC-03	Hỏi đáp pháp luật	Chủ, kế toán, nhân sự
UC-04	Xem và đổi tên hội thoại	Người sở hữu hội thoại
UC-05	Đọc văn bản theo cây Chương–Điều	Chủ, kế toán, nhân sự
UC-06	Tìm chữ trong corpus	Chủ, kế toán, nhân sự
UC-07	Tải hợp đồng và chạy soát xét	Người thuộc một công ty
UC-08	Đọc kết quả soát xét	Người cùng công ty
UC-09	Xem lịch, đánh dấu xong	Người thuộc một công ty
UC-10	Nhập thêm văn bản, dựng lại chỉ mục	Quản trị
UC-11	Khóa / đổi vai trò user	Quản trị
UC-12	Chạy hàng loạt câu hỏi	Quản trị

3.4.1 Đặc tả bổ sung UC-05 và UC-09

3.5 Bảy bước hỏi đáp

Đồ thị LangGraph gồm bảy nút; tên trong mã nguồn giữ tiếng Anh để khớp mã hiện thực. Thứ tự chạy cố định, trừ hai nhánh thoát.

Hệ thống trợ lý pháp lý



Hình 3.2: Sơ đồ use case. Chủ / kế toán / nhân sự dùng bốn nghiệp vụ; quản trị thêm corpus và chạy hàng loạt câu hỏi.

Bảng 3.5: Đặc tả UC-03 — Hỏi đáp.

Trường	Nội dung
Mã	UC-03
Tóm tắt	Người dùng gửi một câu tiếng Việt, nhận câu trả lời kèm Điều dẫn.
Trước khi làm	Đã đăng nhập. Chỉ mục vector sẵn thì hỏi được; chưa sẵn thì nhận 503 và được hướng sang trang tra cứu.
Sau khi làm	Câu hỏi và câu trả lời nằm trong hội thoại, kèm JSON trích dẫn và vết các bước.
Luồng chính	1. Gửi câu. 2. Máy tạo hội thoại nếu chưa có, lấy vài lượt cũ. 3. Ghi câu hỏi. 4. Chạy bảy bước, mỗi bước một sự kiện SSE. 5. Ghi câu trả lời khi có sự kiện kết quả. 6. Trình duyệt đổi URL sang /chat/{id}.
Nhánh	Câu không phải pháp luật: thoát ở bước ý định, không tồn truy hồi. Không tìm ra Điều: nói không có căn cứ, không bịa.
Lỗi	Mất kết nối: câu hỏi đã lưu, gửi lại được. Hết hạn mức mô hình: báo lỗi rõ, không xóa hội thoại. Quá số lần gọi/phút: 429.

Bảng 3.6: Đặc tả UC-07 — Soát xét hợp đồng.

Trường	Nội dung
Mã	UC-07
Tóm tắt	Tải file, chạy soát xét nền, đọc bảng rủi ro.
Trước khi làm	Đã gắn công ty. File .pdf, .docx hoặc .txt, ≤ 10 MB.
Sau khi làm	Tài liệu done hoặc failed. Có thể chạy lại.
Luồng chính	1. Tải, trích chữ, trạng thái ready. 2. Bấm soát xét, trả 202 ngay. 3. Nền: cắt điều khoản, mỗi khoản đi truy hồi, chấm rủi ro. 4. Trang hỏi lại mỗi 2,5 giây.
Nhánh	PDF ảnh (không có lớp chữ): fail lúc tải, không lúc soát xét, để người dùng biết ngay file không trích được chữ. Quá nhiều điều khoản: cắt theo ngưỡng cấu hình, ghi trong tóm tắt.
Lỗi	Sai định dạng: 400. Mất mô hình giữa chừng: failed, bấm thử lại.

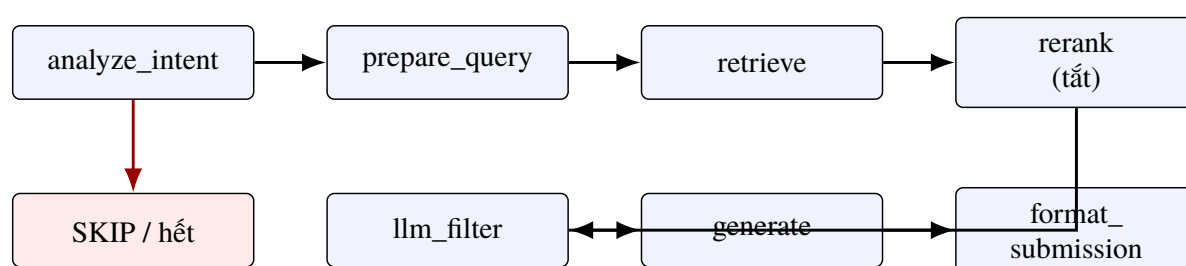
Bảng 3.7: Đặc tả UC-05 — Tra cứu và đọc văn bản.

Trường	Nội dung
Mã	UC-05
Tóm tắt	Duyệt cây Chương–Điều, tìm chữ (kể cả không dấu), mở viện dẫn chéo.
Trước khi làm	Đã đăng nhập. Corpus đã nạp Postgres (không cần Chroma).
Sau khi làm	Người dùng đọc được nguyên văn Điều; có thể chuyển sang hỏi đáp.
Luồng chính	1. Mở /laws. 2. Lọc lĩnh vực hoặc chọn số hiệu. 3. Xem cây. 4. Mở Điều. 5. (Tuỳ chọn) bấm viện dẫn hoặc “Hỏi AI về Điều này”.
Nhánh tìm	Gõ ô tìm: ưu tiên số Điều; rồi FTS; rồi trigram nếu không hit.
Lỗi	Số hiệu sai đường dẫn: 404. Query rỗng: không gọi API.

Bảng 3.8: Đặc tả UC-09 — Lịch tuân thủ.

Trường	Nội dung
Mã	UC-09
Tóm tắt	Xem, đánh dấu, sinh lại các mốc nghĩa vụ theo hồ sơ công ty.
Trước khi làm	User thuộc một organization có vat_period, số lao động.
Sau khi làm	Task phản ánh đúng rule áp dụng; việc đã tick không bị xóa khi sinh lại.
Luồng chính	1. Đăng ký/cập nhật hồ sơ. 2. Hệ thống sinh cửa sổ 3 tháng trước + 12 tháng tới. 3. User xem theo tháng, đánh dấu xong/bỏ qua. 4. Đổi hồ sơ thì bấm sinh lại.
Nhánh	Công ty < 10 lao động: không có rule ATVSLĐ. GTGT tháng/quý loại trừ lẫn nhau.
Lỗi	Chưa gắn công ty: API từ chối. Rule thiếu legal_refs: không được seed.

1. `analyze_intent`. Mô hình chỉ được trả NEXT hoặc SKIP. Câu chào, hỏi giờ, hỏi “ăn gì” dừng tại đây, không tốn nhúng. Khi chạy hàng loạt câu hỏi (trang quản trị), nút này luôn NEXT để mọi câu đều đi truy hồi.
2. `prepare_retrieval_query`. Cấu hình sản phẩm: HyDE bật, viết lại truy vấn nhiều biến thể tắt. Bật đồng thời cả hai làm tăng độ trễ và hạn mức, trong khi HyDE đã đủ để kéo Điều đúng giọng luật. Nếu gọi HyDE lỗi (hết hạn mức, timeout), nút này nuốt lỗi và tìm bằng câu gốc — không được phép làm hỏng cả lượt hỏi chỉ vì một đoạn giả.
3. `retrieve`. BM25 và Chroma chạy song song, gộp RRF, $k_{\text{RRF}} = 60$, trọng số dense/BM25 = 3/1, lấy top $k = 8$ Điều.
4. `rerank`. Trong file cấu hình đang tắt vì Gemini không có cổng rerank riêng; nút vẫn nằm trên đồ thị để bật được khi tự host mô hình rerank.
5. `llm_filter`. Tối đa 3 Điều lọc song song, giữ tối thiểu 2 (tránh lọc sạch rồi không còn gì để viết). Mỗi ứng viên được hỏi “có dính câu hỏi này không”, chỉ được trả có hoặc không.
6. `generate_answer`. Nhiệt độ 0, tối đa 4096 token, stream từng token xuống SSE. Prompt hệ thống siết: chỉ kết luận từ căn cứ đang có; thiếu căn cứ thì nói thiếu, không bịa mức phạt.
7. `format_submission`. Regex lấy các cụm “Điều n ” trong câu trả lời, đối chiếu với danh sách id vừa truy hồi. Số không có trong danh sách bị gỡ khỏi mảng citation gửi xuống giao diện. Nội dung câu trả lời được giữ nguyên để tránh cắt câu; người dùng chỉ không bấm được vào nguồn không thuộc tập truy hồi.



Hình 3.3: Bảy nút LangGraph: nhánh SKIP thoát sớm; rerank tắt trên sản phẩm Gemini.

3.6 Corpus

Kho nằm trong `data/base_data.json`: 3.844 bản ghi. Tập kê khai `corpus/law_manifest.json` gán lĩnh vực cho từng số hiệu, dùng để đổ bảng laws mỗi lần backend lên.

3.6.1 Nguồn ngữ liệu

Đồ án không gửi request hàng loạt lên cổng văn bản quy phạm. Cả vbpl.vn và thuvienphapluat.vn đều chặn truy cập tự động. Nội dung được lấy từ bộ dữ liệu công khai pdt590/vietnamese-legal-documents trên Hugging Face [21]: bản sao dạng markdown của thư viện pháp luật, khoảng 518 nghìn văn bản. Script corpus/build_corpus.py chỉ giữ các số hiệu có trong manifest, đọc metadata và mười một phần parquet, kiểm tra đúng văn bản (từ chối nếu số Điều quá ít — đã gặp trường hợp mirror trả nhầm văn bản), rồi cắt thành từng Điều và ghi JSON.

Hai hệ quả. Thứ nhất, từng Điều vẫn truy được về số hiệu và URL gốc (lưu trong báo cáo dựng kho). Thứ hai, chất lượng kho phụ thuộc bản sao: script không thay luật sư đổi chiều hiệu lực. Việc chọn bản còn hiệu lực khi hai số hiệu cùng tên khác năm chưa được tự động hóa.

Bảng 3.9: Đếm nhanh theo loại và theo lĩnh vực (một văn bản vào đúng một lĩnh vực).

Loại	Số	Lĩnh vực	Số
Luật	19	Thuế	12
Nghị định	14	Lao động	4
Thông tư	3	Sở hữu trí tuệ	4
Bộ luật	2	Doanh nghiệp	3
		Dân sự – hợp đồng	3
		Kế toán – tài chính	3
		Còn lại (BHXH, ATVSLĐ, NTD, đất đai, hỗ trợ DNNVV)	9
Cộng	38	Cộng	38

Thuế nhiều nhất vì lịch tuân thủ và câu hỏi hàng ngày của kế toán tập trung ở đó. Có cả luật cũ và luật mới cùng tên (ví dụ Luật GTGT 13/2008 và 48/2024): hệ thống giữ cả hai số hiệu vì hợp đồng thực tế vẫn có thể dẫn bản hiệu lực tại thời điểm ký. Việc chọn tự động bản còn hiệu lực chưa được hiện thực và được ghi nhận là hạn chế.

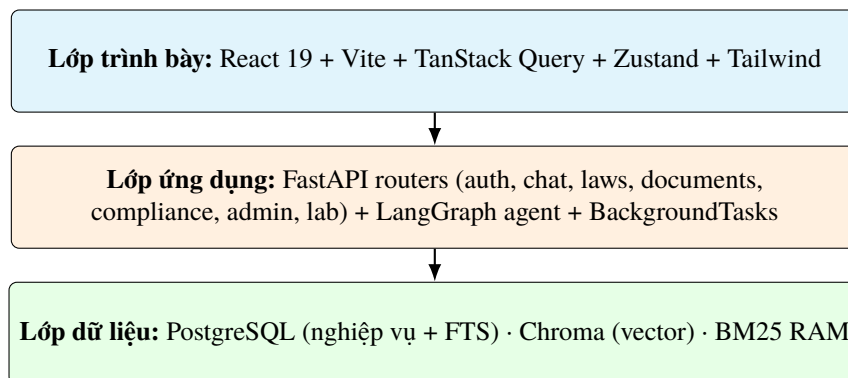
Cắt theo Điều đã nói ở Chương 2. Trường extra giữ viện dẫn. Bảng legal_knowledge_records giữ khóa id nguyên từ file JSON, không tự tăng, để Chroma và Postgres chỉ cùng một mã.

3.7 Mô hình dữ liệu

3.7.1 Một số quyết định trên lược đồ dữ liệu

Thứ tự tin nhắn. Câu hỏi và câu trả lời được ghi trong cùng một giao dịch HTTP. Hàm now() của PostgreSQL trả về thời điểm bắt đầu giao dịch, nên hai dòng có thể trùng created_at và thứ tự khi đọc lại không ổn định. Lược đồ dùng cột seq kiểu identity, API sắp xếp theo seq.

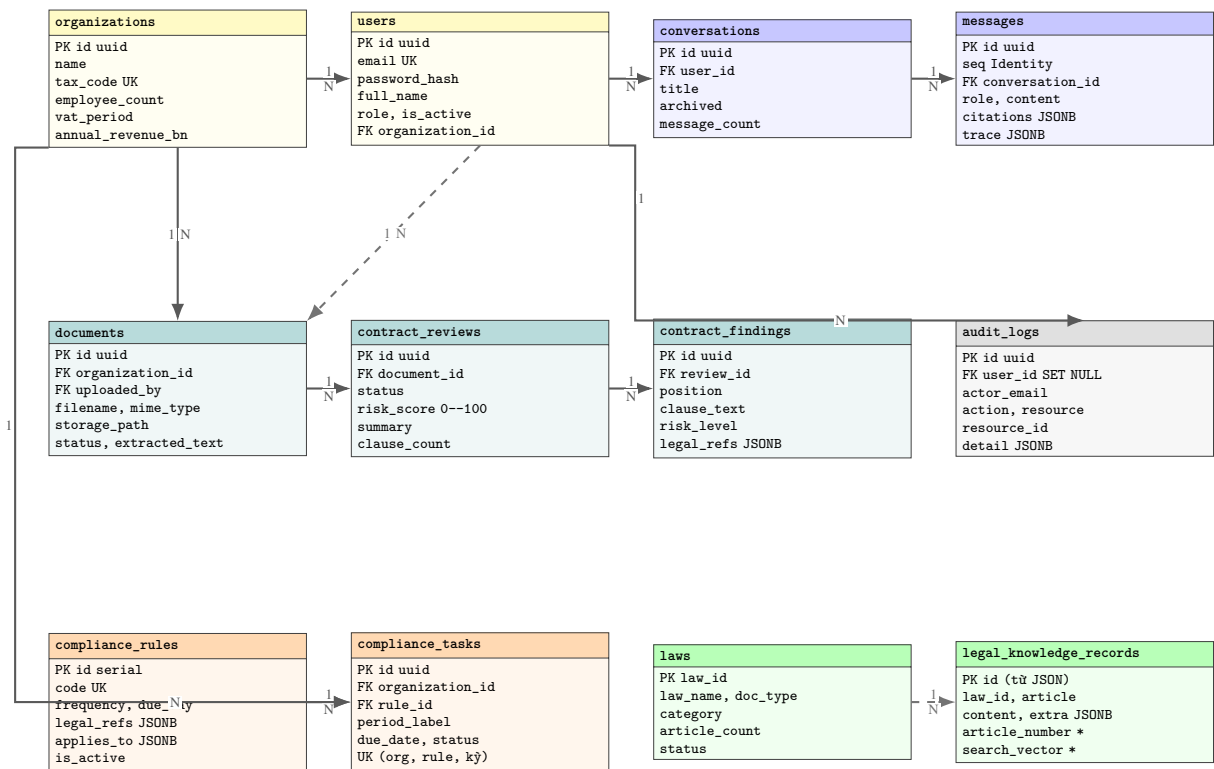
Mã lỗi khi truy cập chéo hội thoại. Trả 403 khi tài nguyên tồn tại nhưng không thuộc người dùng sẽ tiết lộ sự tồn tại của định danh. Hệ thống trả 404, cùng hành vi với trường hợp



Hình 3.4: Kiến trúc ba lớp: trình bày, ứng dụng, dữ liệu.

Bảng 3.10: Các bảng nghiệp vụ và ghi chú hiện thực.

Bảng	Ghi chú hiện thực
organizations	vat_period, employee_count quyết định rule nào được sinh. tax_code unique.
users	Email lower-case. Role text, không dùng ENUM native Postgres để khỏi ALTER TYPE mỗi lần thêm giá trị.
conversations	Tiêu đề cắt từ câu hỏi đầu, tối đa 80 ký tự, cắt tại khoảng trắng.
messages	Cột seq identity. Không sort theo created_at.
documents	File trên đĩa tên UUID, không lấy tên gốc.
contract_reviews, contract_findings	Một tài liệu nhiều lượt soát xét, không ghi đè.
compliance_rules / tasks	Unique (công ty, rule, kỳ) để sinh lại không nhân bản.
legal_knowledge_ records	Generated: article_number, doc_ref, search_vector.
laws	Đổ từ manifest + đếm Điều, mỗi lần startup.
audit_logs	Đăng ký, đăng xuất, đổi mật khẩu, tải file, reindex.



Nét liền: khóa ngoại thật (CASCADE hoặc SET NULL). Nét đứt: uploaded_by và laws ↔ legal_knowledge_records (cùng law_id, không FK).
 * = cột generated (Postgres tự tính). UK = unique. Màu: vàng tài khoản, xanh dương hội thoại, teal hợp đồng, cam lịch, xanh lục corpus, xám nhật ký.

Hình 3.5: Lược đồ quan hệ PostgreSQL (12 bảng). Hội thoại thuộc user; hợp đồng và lịch thuộc organization. Corpus cắt theo Điều; laws là catalog đếm từ cùng law_id.

không có bản ghi.

Bộ nhớ hội thoại của LangGraph. InMemorySaver lưu trạng thái trên RAM, mất khi khởi động lại và không chia sẻ giữa nhiều worker. Hệ thống nạp N lượt gần nhất từ PostgreSQL vào state; `short_memory.max_turns = 6`.

3.7.2 Chi tiết từng bảng và khóa ngoại

Mục này mô tả bảng PostgreSQL phía sản phẩm (không kể bảng Alembic version). Corpus tra cứu dùng `legal_knowledge_records`; catalog có thể có bảng `laws` tùy migration. Vector không nêu ở đây.

3.7.3 organizations và users

`organizations`: tên, mã số thuế unique, số lao động, kỳ GTGT, tùy chọn doanh thu. Xóa tổ chức cascade tài liệu và task; user bị SET NULL khỏi tổ chức để không mất tài khoản.

`users`: email unique (lower-case khi ghi), `password_hash`, role kiểu text, `is_active`. Khóa user không xóa hội thoại ngay — tùy chính sách admin. UUID làm khóa chính, không lộ số tăng dần.

3.7.4 conversations và messages

Hội thoại thuộc đúng một user, có tiêu đề, cờ lưu trữ, đếm tin nhắn. `messages` có seq Identity: cùng giao dịch, `created_at` của Postgres có thể trùng, sắp theo thời gian làm đảo câu hỏi/đáp. JSONB lưu citation và vết pipeline cho tin assistant. Xóa hội thoại cascade tin nhắn.

3.7.5 legal_knowledge_records

Mỗi hàng một Điều. Khóa id lấy từ corpus, không autoincrement, để id ổn định giữa lần nạp. Cột generated: `article_number` (số để sắp 9 trước 10), `doc_ref`, `search_vector` (tsvector đã unaccent, trọng số A/B/C theo tiêu đề / tên luật / nội dung). JSONB extra: viện dẫn chéo. Chỉ mục: (`law_id`, `article`), GIN tsvector, GIN trigram tiêu đề và tên luật.

3.7.6 compliance_rules và compliance_tasks

Rule: mã unique, tần suất, công thức hạn, `legal_refs`, `applies_to`, `is_active`. Task: tổ chức, rule, nhãn kỳ, ngày đến hạn, trạng thái, unique (tổ chức, rule, kỳ). Index theo hạn để lịch tháng không quét full.

3.7.7 documents, contract_reviews, contract_findings

Document: file trên volume, MIME, kích thước, chữ trích, trạng thái. Review: điểm rủi ro 0–100, tóm tắt, số điều khoản. Finding: vị trí, mức, gợi ý, căn cứ. Xóa tài liệu cascade review và finding. Đường file không phải URL công khai; tải về đi qua API đã xác thực.

3.7.8 Nhật ký audit (nếu có trong schema)

Bảng audit ghi hành động admin (reindex, khóa user) khi module được bật, phục vụ truy vết, không phục vụ RAG. Không ghi mật khẩu hay token.

Bảng 3.11: Bảng PostgreSQL chính và khóa ngoại then chốt.

Bảng	Liên kết / ràng buộc
users	organization_id → organizations, SET NULL
conversations	user_id CASCADE
messages	conversation_id CASCADE; seq Identity
legal_knowledge_records	PK id từ corpus; GIN search
compliance_tasks	unique (org, rule, period)
documents	org CASCADE; uploaded_by SET NULL
contract_findings	review CASCADE

3.8 Tìm chữ trên tiếng Việt

Hàm unaccent mặc định là STABLE, không dùng trong generated column. Hàm immutable_unaccent được bọc trong init.sql, chạy bằng superuser lúc tạo cụm, Alembic không dụng tới (user ứng dụng không đủ quyền CREATE EXTENSION).

Ba tầng tìm, theo thứ tự:

1. Câu có dạng “Điều 12” thì lọc thẳng article_number.
2. tsquery kiểu prefix (doanh:*) trên search_vector, xếp ts_rank_cd.
3. Nếu không ra dòng nào, pg_trgm với ngưỡng 0,3 trên tiêu đề Điều và tên văn bản — cứu gở sai.

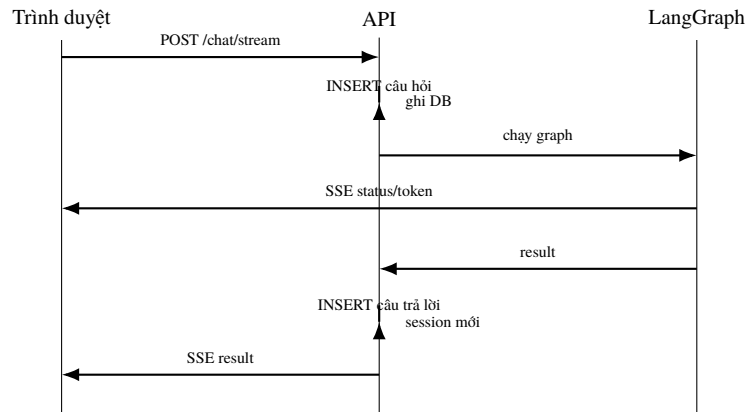
Hàm ts_headline chạy trên cột đã bỏ dấu, trả về đoạn không dấu nên không dùng. Backend gửi list token; React bôi <mark> trên chữ gốc bằng so khớp bỏ dấu phía client.

3.9 Luồng thời gian thực và lưu hội thoại

Ba loại sự kiện: status, token, result. Có heartbeat khi một bước kéo dài, để proxy khởi cắt kết nối. Client không dùng EventSource.

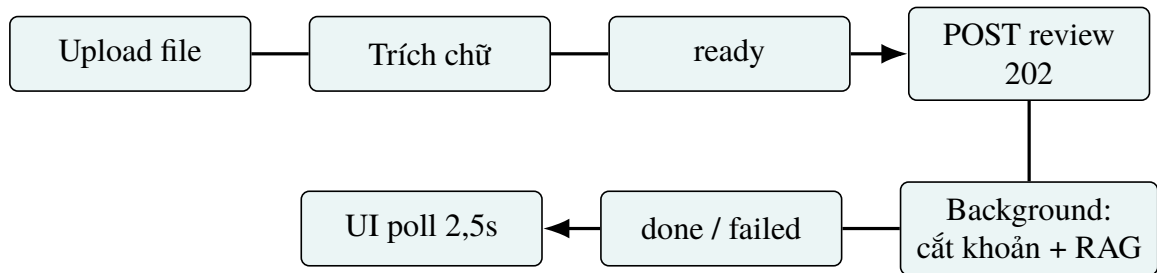
3.10 Hợp đồng

Cắt điều khoản bằng regex tiêu đề “Điều n”, “Article n”, hoặc dòng số kiểu “5. Thanh toán”. Ưu tiên đuôi file vì trình duyệt hay khai application/octet-stream. Khoản dưới 80 ký tự ghép

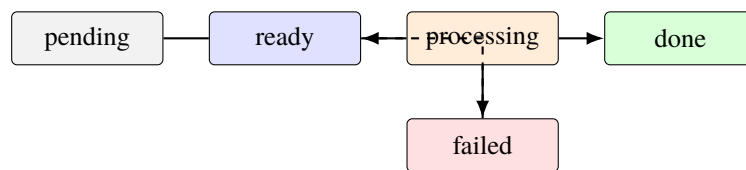


Hình 3.6: Luồng hỏi đáp: ghi câu hỏi trước, stream SSE, ghi câu trả lời ở session CSDL riêng.

vào khoản trước — thường là tiêu đề mục. Khoản dài quá 6000 ký tự bị cắt để khỏi vượt ngữ cảnh. Tối đa 60 khoản một lượt; hợp đồng mẫu DNNVV dùng khi thử nghiệm nằm dưới ngưỡng này. Mỗi khoản gọi lại cùng hàm truy hồi của hỏi đáp, lấy 4 Điều, rồi mô hình gán mức cao / trung bình / thấp / thông tin. Điểm tổng 0–100 từ trọng số mức. PDF không có lớp chữ (scan) fail lúc tải, có câu tiếng Việt bảo cần OCR — không đợi đến lúc bấm soát xét mới báo.



Hình 3.7: Luồng soát xét hợp đồng: trả 202 ngay, xử lý nền, UI poll trạng thái.



Hình 3.8: Máy trạng thái tài liệu: pending → ready → processing → done/failed.

Lúc viết frontend, Axios gán sẵn Content-Type: application/json, gửi FormData làm hỏng biên multipart. Interceptor gỡ header đó khi body là FormData. Đây là lỗi chỉ lộ khi bấm tải file trên trình duyệt; gọi API thuần JSON thì không dính.

3.11 Lịch tuân thủ

12 rule nằm trong mã nguồn Python, được seed mỗi lần backend khởi động. Đồ án không nhét hạn chót vào file cấu hình YAML: rule gán legal_refs (số hiệu + tên văn bản + số Điều). Nếu một hạn chót đổi vì luật mới, người sửa phải đổi cả refs, không được chỉ sửa ngày.

Bảng 3.12: Mười hai rule tuân thủ và điều kiện hồ sơ để rule được sinh.

Mã	Kỳ	Điều kiện / hạn chót
VAT_MONTHLY	Tháng	Chỉ khi khai GTGT tháng; ngày 20 tháng sau
VAT_QUARTERLY	Quý	Chỉ khi khai quý; cuối tháng đầu quý sau
CIT_PROVISIONAL	Quý	Mọi công ty; ngày 30 tháng đầu quý sau
CIT_FINALIZATION	Năm	Cuối tháng 3
PIT_FINALIZATION	Năm	Có từ 1 lao động; cuối tháng 3
BUSINESS_LICENSE_FEE	Năm	30/01
SOCIAL_INSURANCE_MONTHLY	Tháng	Có lao động; cuối tháng
LABOUR_REPORT_H1 / H2	Năm	05/06 và 05/12
LABOUR_SAFETY_REPORT	Năm	Từ 10 lao động; 10/01 năm sau
FINANCIAL_STATEMENT	Năm	Cuối tháng 3
INVOICE_USAGE_CHECK	Quý	Rà hóa đơn điện tử, ngày 15 tháng sau quý

Sinh của số 3 tháng trước (để thấy việc đã quá hạn) và 12 tháng tới. Khóa duy nhất (công ty, rule, kỳ) nên bấm “Sinh lại lịch” sau khi đổi số lao động không xóa việc đã đánh dấu xong. Công ty 12 người, khai quý: khi đăng ký hệ thống sinh 39 task — có GTGT quý, không có GTGT tháng, có báo cáo an toàn lao động vì đủ ngưỡng 10 người. Công ty 3 người khai tháng thì đảo lại: có rule tháng, không có báo cáo an toàn. Test backend bắt đúng hai nhánh này.

Hạn chế: hạn chót lấy từ Điều trong corpus tại thời điểm thu thập. Luật sửa sau đó mà chưa nhập văn bản mới thì lịch vẫn nhắc ngày cũ. Hệ thống không tự đọc công báo.

3.12 Đăng nhập, phiên làm việc, bảo mật

Mật khẩu băm argon2id (passlib). JWT thuật toán HS256, hai loại token, payload có sub, type, iat, exp, jti. Access sống 15 phút, refresh 7 ngày. Gọi API bằng refresh token bị từ chối vì type không khớp. Interceptor phải phân biệt hai loại token; đúng ra lỗi đó.

Tài khoản đăng ký đầu tiên, khi bảng users còn trống, được gán admin. Cờ bootstrap_first_admin bật mặc định. Không có tài khoản “admin/admin” cứng trong image, nhằm tránh mật khẩu mẫu nằm trong Docker Hub sau này.

Phía trình duyệt, Axios gán sẵn Authorization. Gặp 401, một lần refresh được chia cho mọi request đang chờ (một Promise chung), tránh gọi /auth/refresh song song rồi làm hỏng phiên. Endpoint auth không đi vào vòng refresh, kéo login sai mật khẩu cũng bị “làm mới token”. Token nằm localStorage: phù hợp quy mô đồ án nhưng còn rủi ro XSS; CSP và cookie HttpOnly chưa được hiện thực.

CORS: danh sách origin, allow_credentials=true nên không được *. Trên Docker, nginx và API cùng origin nên CORS gần như không chạy; danh sách chủ yếu phục vụ Vite lúc

lập trình (localhost:5173).

Giới hạn tần suất: chat 20 lần/phút, tải file 30 lần/giờ, theo từng user (hoặc IP nếu chưa đăng nhập). User thường gọi /api/v1/admin/* nhận 403. Hội thoại người khác: 404, đã nói ở trên.

Bốn vai trò owner, accountant, hr, admin. Hợp đồng và lịch tuân thủ cách ly theo organization_id. Hội thoại cách ly theo user_id — kế toán và chủ cùng công ty không đọc được chat của nhau. Thiết kế này phù hợp câu hỏi pháp lý mang tính riêng (lương, kỷ luật).

3.13 Giao diện

Frontend là một SPA. Người chưa đăng nhập chỉ vào /login và /register. Người đã đăng nhập bị đẩy khỏi hai trang đó. Route cần phiên chờ restore() đọc token xong mới quyết định, nếu không sẽ nhảy sang login rồi nhảy lại.

Số hiệu văn bản kiểu 45/2019/QH14 chứa dấu gạch chéo. React Router mặc định cắt path theo /, nên định tuyến bắt phần còn lại bằng /laws/:lawId/*. Backend cũng nhận path, không nhận query một đoạn, vì cùng lý do.

Trang quản trị lazy() tách bundle: user thường không tải mã corpus và trang chạy hàng loạt. TanStack Query tắt refetchOnWindowFocus — corpus không đổi mỗi lần đổi tab, refetch chỉ tổ làm nhấp nháy cây Điều.

Ảnh chụp bố cục từng trang (hỏi đáp, tra cứu, hợp đồng, lịch) đặt ở Chương 4 khi đánh giá giao diện, tránh trùng hình trong báo cáo.

Trang hồ sơ cho sửa số lao động và kỳ GTGT; lưu xong lịch được sinh lại. Trang chạy hàng loạt: tải file JSON danh sách câu, xem tiến độ, tải kết quả JSON.

Giao diện dùng thành phần tự viết để giảm kích thước gói. Nền tối, chữ đủ tương phản để đọc Điều dài. Bố cục chưa tối ưu cho điện thoại; thanh hội thoại trên màn hẹp còn chật — đây là hạn chế đã ghi nhận.

3.14 Nhúng vector, cache BM25 và biến môi trường

Mỗi lần đổi model nhúng hoặc đổi số chiều, backend so manifest trong thư mục Chroma với cấu hình hiện tại. Lệnh thì nhúng lại. Lô 64 Điều, nghỉ dần khi gặp 429 (đợi 2 giây, nhân đôi, trần 60 giây, tối đa 6 lần). Gói miễn phí Google dễ đụng trần nếu nhúng 3.844 Điều một mạch; không có backoff thì lần đầu cài gần như chắc fail giữa chừng.

BM25 tách từ bằng underthesea (fallback regex). Kết quả ghi file cache theo dấu vân tay tokenizer. 3.844 Điều tách lại mỗi lần restart thì startup kéo dài hàng phút; cơ chế cache được bổ sung sau khi đo thời gian khởi động.

Docker Compose chỉ đọc .env ở thư mục gốc dự án. File backend/.env dùng khi lập triển khai độc lập **không** được compose nạp. Khóa API phải đặt tại file gốc; hướng dẫn được nêu trong README và Phụ lục A.

Thiếu khóa: healthcheck vẫn xanh, /health trả ổn, đăng nhập và tra cứu chạy. Hỏi đáp trả

503 tiếng Việt, gợi ý dùng trang luật. Đây là chỗ thầy góp ý sớm; phiên bản đầu tiên từng raise lúc import embedding client, kéo theo cả cụm không lên.

3.15 Đóng gói ba dịch vụ

Ba container trên một mạng cầu. Postgres 17, healthcheck pg_isready, volume riêng, init.sql tạo extension unaccent, pg_trgm và hàm immutable_unaccent. Backend đợi Postgres healthy rồi mới lên, healthcheck HTTP /health, start_period 60 giây vì lần đầu Alembic và nạp catalog luật chậm. Frontend là nginx phục vụ bản Vite đã build, proxy /api sang backend, VITE_API_BASE_URL để trống để SPA gọi đường dẫn tương đối — cùng origin, hết chuyện CORS trên môi trường cài đặt.

Cổng ra máy host: web 5173, API 8023, Postgres 23432 (không 5432, tránh đụng Postgres sẵn có trên máy phát triển). Corpus và data/ mount read-only. chroma_db và uploads là volume, sống sót khi rebuild image.

3.16 Tóm tắt hiện thực theo module

Bảng 3.13: Ánh xạ module code ↔ nghiệp vụ.

Thư mục / router	Việc
routers/auth	Đăng ký, login, refresh, đổi mật khẩu
routers/legal	+ SSE hỏi đáp, bấm nút LangGraph
agents/ routers/laws	+ Catalog, cây, FTS ba tầng
legal/search	
routers/documents	+ Upload, trích chữ, soát xét nền
contracts/ routers/compliance	12 rule, sinh task
+ seed	
routers/admin, lab	Corpus, user, chạy hàng loạt câu hỏi
frontend/pages/*	Chat, Laws, Contracts, Compliance, Admin, bộ câu hỏi

Các kịch bản hỏi đáp và chạy cụm trình bày ở Chương 4 — chương này chỉ chốt thiết kế và quyết định hiện thực.

3.17 Thiết kế API, cấu hình và các khối hiện thực

3.17.1 API theo tài nguyên

- **Là gì.** REST dưới tiền tố /api/v1: /auth, /conversations, /legal/chat/stream, /laws, /documents, /compliance, /admin, /health.

- **Vì sao dùng.** Tách tài nguyên khớp bốn nghiệp vụ; v1 để sau này thêm phiên bản không phá client cũ.
- **Cách dùng.** GET đọc, POST tạo, PATCH sửa, DELETE xóa. Lỗi JSON detail. /health không cần token.
- **Khi nào.** Mọi request nghiệp vụ. SSE chỉ trên /legal/chat/stream.

3.17.2 Đăng ký và bootstrap quản trị

- **Là gì.** Tài khoản đầu (bảng user trống, cờ bootstrap_first_admin) nhận admin; các tài khoản sau là owner nếu kèm tổ chức.
- **Vì sao dùng.** Không nhúng cặp admin/admin trong image.
- **Cách dùng.** Đăng ký có thể kèm MST, số lao động, kỳ GTGT; ngay sau đó sinh lịch.
- **Khi nào.** Chỉ lần đăng ký đầu trên CSDL trống. Các lần sau không tự phong admin.

3.17.3 Cấu hình config.yaml

- **Là gì.** File YAML mô tả hành vi pipeline và ngưỡng; không chứa bí mật.
- **Vì sao dùng.** Đổi HyDE/rewrite/rerank/filter không sửa mã. Bí mật đi biến môi trường.
- **Cách dùng.** Nhóm llm, embeddings, legal_assistant.*, vector_store (hybrid, $k = 8$, RRF 60, trọng số 3:1), short_memory.max_turns=6, auth, rate_limit, uploads (10 MB).
- **Khi nào.** Mỗi lần backend khởi động. Đổi chiều embedding: xóa volume Chroma rồi nhúng lại.

3.17.4 Tách chữ và soát xét hợp đồng

- **Là gì.** Trích chữ PDF (lớp văn bản), DOCX (XML), TXT (UTF-8); cắt theo tiêu đề điều khoản; mỗi khoản đi cùng hàm truy hồi rồi LLM gán mức rủi ro.
- **Vì sao dùng.** Tái sử dụng retrieval: căn cứ hợp đồng cũng mở được Điều, cùng hàng rào với chat. Không huấn luyện mô hình phát hiện rủi ro riêng.
- **Cách dùng.** Regex “Điều n ”, “Article n ”, dòng số. Gộp khoản < 80 ký tự; cắt khoản > 6000 . Tối đa 60 khoản, 3 khoản song song. $k = 4$ Điều mỗi khoản. Điểm 0–100 từ trọng số cao/trung/thấp.
- **Khi nào.** Trích chữ lúc tải (201/ready). Soát xét lúc user bấm (202, nền). PDF ảnh: fail lúc tải, không đợi soát xét.

3.17.5 Giao diện từng trang

- **Là gì.** SPA: login/register, chat, luật, hợp đồng, lịch, hồ sơ, admin (corpus, user, chạy hàng loạt).
- **Vì sao dùng.** Mỗi nghiệp vụ một nhóm route; citation là nút mở Điều, không phải footnote tĩnh.
- **Cách dùng.** AppShell ẩn mục admin nếu không phải quản trị. Số hiệu có / dùng splat.
- **Khi nào.** Sau đăng nhập. Login/register khi chưa có phiên.

3.17.6 Nhật ký và chiến lược lỗi

- **Là gì.** audit_logs ghi đăng ký, đăng xuất, đổi mật khẩu, tải tệp, reindex. Thông báo lỗi tiếng Việt, không traceback.
- **Vì sao dùng.** Truy vết khi nhầm reindex; người dùng biết thiếu index hay file không có chữ.
- **Cách dùng.** 503 gợi ý tra cứu khi chưa nhúng. Hết hạn mức: giữ câu hỏi đã lưu. Mất SSE: gửi lại được.
- **Khi nào.** Mỗi hành động nhạy cảm (ghi log). Mọi lỗi trả về client (thông báo). Chưa có UI xem log.

3.18 Danh mục văn bản trong corpus

- **Là gì.** Bảng 3.14: đủ 38 số hiệu đã nạp, mỗi dòng một lĩnh vực theo manifest.
- **Vì sao dùng.** Người đọc đối chiếu được kho thật, không chỉ con số “38 luật”.
- **Cách dùng.** Tên rút gọn nếu dài; tên đầy đủ nằm bảng laws.
- **Khi nào.** Khi đọc báo cáo và khi admin mở kho. Không phải danh mục VBQPPL toàn quốc.

Bảng 3.14: Danh mục 38 văn bản quy phạm trong corpus.

Số hiệu	Loại	Lĩnh vực	Tên
04/2017/QH14	Luật	Hỗ trợ DNNVV	Luật Hỗ trợ doanh nghiệp nhỏ và vừa

Bảng 3.14: Danh mục văn bản (tiếp).

Số hiệu	Loại	Lĩnh vực	Tên
80/2021/NĐ-CP	Nghị định	Hỗ trợ DNNVV	Nghị định Quy định chi tiết và hướng dẫn thi hành một số điều của Luật Hỗ trợ doanh ngh...
59/2020/QH14	Luật	Doanh nghiệp	Luật Doanh nghiệp
01/2021/NĐ-CP	Nghị định	Doanh nghiệp	Nghị định về đăng ký doanh nghiệp
122/2021/NĐ-CP	Nghị định	Doanh nghiệp	Nghị định Quy định về xử phạt vi phạm hành chính trong lĩnh vực kế hoạch và đầu tư
38/2019/QH14	Luật	Thuế	Luật Quản lý thuế
108/2025/QH15	Luật	Thuế	Luật Quản lý thuế
126/2020/NĐ-CP	Nghị định	Thuế	Nghị định Quy định chi tiết một số điều của Luật Quản lý thuế
123/2020/NĐ-CP	Nghị định	Thuế	Nghị định Quy định về hóa đơn, chứng từ
78/2021/TT-BTC	Thông tư	Thuế	Thông tư Hướng dẫn thực hiện một số điều của Luật Quản lý thuế ngày 13 tháng 6 năm 2019...
132/2020/NĐ-CP	Nghị định	Thuế	Nghị định Quy định về quản lý thuế đối với doanh nghiệp có giao dịch liên kết
139/2016/NĐ-CP	Nghị định	Thuế	Nghị định Quy định về lệ phí môn bài
13/2008/QH12	Luật	Thuế	Luật Thuế giá trị gia tăng
48/2024/QH15	Luật	Thuế	Luật Thuế giá trị gia tăng
14/2008/QH12	Luật	Thuế	Luật Thuế thu nhập doanh nghiệp
67/2025/QH15	Luật	Thuế	Luật Thuế thu nhập doanh nghiệp
04/2007/QH12	Luật	Thuế	Luật Thuế thu nhập cá nhân
45/2019/QH14	Bộ luật	Lao động	Bộ luật Lao động
145/2020/NĐ-CP	Nghị định	Lao động	Nghị định Quy định chi tiết và hướng dẫn thi hành một số điều của Bộ luật Lao động về đ...
12/2022/NĐ-CP	Nghị định	Lao động	Nghị định Quy định xử phạt vi phạm hành chính trong lĩnh vực lao động, bảo hiểm xã hội,...

Bảng 3.14: Danh mục văn bản (tiếp).

Số hiệu	Loại	Lĩnh vực	Tên
152/2020/NĐ-CP	Nghị định	Lao động	Nghị định Quy định về người lao động nước ngoài làm việc tại Việt Nam và tuyển dụng, qu...
41/2024/QH15	Luật	Bảo hiểm xã hội	Luật Bảo hiểm xã hội
58/2020/NĐ-CP	Nghị định	Bảo hiểm xã hội	Nghị định Quy định về mức đóng bảo hiểm xã hội bắt buộc vào Quỹ bảo hiểm tai nạn lao độ...
84/2015/QH13	Luật	An toàn lao động	Luật An toàn, vệ sinh lao động
44/2016/NĐ-CP	Nghị định	An toàn lao động	Nghị định Quy định chi tiết một số điều của Luật An toàn, vệ sinh lao động về hoạt động...
50/2005/QH11	Luật	Sở hữu trí tuệ	Luật Sở hữu trí tuệ
36/2009/QH12	Luật	Sở hữu trí tuệ	Luật sửa đổi, bổ sung một số điều của Luật Sở hữu trí tuệ
07/2022/QH15	Luật	Sở hữu trí tuệ	Luật sửa đổi, bổ sung một số điều của Luật Sở hữu trí tuệ
65/2023/NĐ-CP	Nghị định	Sở hữu trí tuệ	Nghị định Quy định chi tiết một số điều và biện pháp thi hành Luật Sở hữu trí tuệ về sở...
91/2015/QH13	Bộ luật	Dân sự - Hợp đồng	Bộ luật Dân sự
36/2005/QH11	Luật	Dân sự - Hợp đồng	Luật Thương mại
54/2010/QH12	Luật	Dân sự - Hợp đồng	Luật Trọng tài thương mại
88/2015/QH13	Luật	Kế toán - Tài chính	Luật Kế toán
133/2016/TT-BTC	Thông tư	Kế toán - Tài chính	Thông tư Hướng dẫn Chế độ kế toán doanh nghiệp nhỏ và vừa
200/2014/TT-BTC	Thông tư	Kế toán - Tài chính	Thông tư Hướng dẫn Chế độ kế toán doanh nghiệp
19/2023/QH15	Luật	Bảo vệ người tiêu dùng	Luật Bảo vệ quyền lợi người tiêu dùng

Bảng 3.14: Danh mục văn bản (tiếp).

Số hiệu	Loại	Lĩnh vực	Tên
98/2020/NĐ-CP	Nghị định	Bảo vệ người tiêu dùng	Nghị định Quy định xử phạt vi phạm hành chính trong hoạt động thương mại, sản xuất, buôn...
31/2024/QH15	Luật	Đất đai	Luật Đất đai

Danh mục cho thấy thuế chiếm nhiều văn bản nhất, phù hợp lịch tuân thủ và câu hỏi hàng ngày của kế toán. Có cả luật cũ và luật mới cùng tên (ví dụ quản lý thuế, thuế GTGT): hệ thống giữ theo số hiệu, chưa tự lọc bản còn hiệu lực.

3.19 Mười hai nghĩa vụ tuân thủ và cách sinh lịch

- **Là gì.** Bộ 12 bản ghi khai báo trong mã (COMPLIANCE_RULES): mã, tiêu đề, tần suất, hạn chót, `applies_to`, `legal_refs` trỏ Điều trong corpus.
- **Vì sao dùng.** DNNVV cần nhắc mốc thuế, BHXH, lao động, BCTC. LLM không được suy hạn nộp — dễ bịa ngày.
- **Cách dùng.** Bộ sinh đọc hồ sơ tổ chức, tạo task cửa sổ 3 tháng trước + 12 tháng tới. INSERT ...ON CONFLICT DO NOTHING theo (công ty, rule, kỳ).
- **Khi nào.** Lúc đăng ký/cập nhật hồ sơ và khi user bấm sinh lại. Không chạy khi chưa gắn tổ chức. Luật đổi hạn: sửa rule trong mã, không chờ mô hình nhớ.

Mỗi mục dưới đây: nghĩa vụ là gì, công thức hạn trong mã, và khi nào rule được lọc vào hồ sơ.

3.19.1 VAT_MONTHLY — khai nộp GTGT theo tháng

Áp dụng khi hồ sơ khai `vat_period = monthly`. Hạn: ngày 20 của tháng liền sau tháng phát sinh, theo hướng Điều 44 Luật Quản lý thuế (hồ sơ khai) và Điều 55 (thời hạn nộp tiền) số 38/2019/QH14. `due_day = 20`, `due_month_offset = 1`.

Doanh nghiệp nhỏ thường được chọn quý; rule tháng tồn tại vì một số đơn vị vẫn kê khai tháng, và vì demo cần thấy khác biệt khi đổi hồ sơ.

3.19.2 VAT_QUARTERLY — khai nộp GTGT theo quý

Áp dụng `vat_period = quarterly`. Hạn được mã hoá ngày cuối tháng đầu quý sau (`due_day = 31` kết hợp offset một tháng, bộ sinh kẹp theo số ngày thật của tháng). Căn cứ: Điều 44 Luật Quản lý thuế và Điều 8 Nghị định 126/2020/NĐ-CP. Đây là rule người dùng DNNVV gặp nhiều nhất trên lịch.

3.19.3 CIT_PROVISIONAL — tạm nộp thuế TNDN theo quý

Mọi tổ chức trong phạm vi đồ án. Hạn ngày 30 tháng đầu quý sau. Rule nhấn: không phát sinh hồ sơ khai tạm tính quý, chỉ nghĩa vụ tiền. Căn cứ Điều 55 Luật Quản lý thuế và Điều 8 Nghị định 126/2020/NĐ-CP. Nhằm lẫn thường gặp là coi tạm nộp như một tờ khai — mô tả rule ghi rõ để UI không gọi là “khai quý TNDN”.

3.19.4 CIT_FINALIZATION — quyết toán TNDN năm

Hạn ngày cuối tháng thứ ba sau khi kết thúc năm dương lịch ($\text{due_fixed_month} = 3, \text{due_day} = 31$). Căn cứ Điều 44 Luật Quản lý thuế và Điều 12 Luật Thuế TNDN 14/2008/QH12 (kho còn số hiệu này; luật mới cùng chủ đề được nạp riêng, rule chưa tự chuyển). Năm tài chính khác dương lịch chưa tham số hoá — hạn chế phạm vi.

3.19.5 PIT_FINALIZATION — quyết toán TNCN năm

Chỉ khi $\text{min_employees} \geq 1$. Tổ chức trả thu nhập quyết toán thay người lao động, hạn cùng cửa sổ tháng thứ ba. Căn cứ Điều 44 Luật Quản lý thuế và Điều 24 Luật Thuế TNCN 04/2007/QH12. Công ty không có lao động không thấy task này.

3.19.6 BUSINESS_LICENSE_FEE — lệ phí môn bài

Hạn 30 tháng 01 hằng năm. Căn cứ Điều 5 Nghị định 139/2016/NĐ-CP và Điều 10 Nghị định 126/2020/NĐ-CP. Rule không tính số tiền theo vốn điều lệ — đồ án nhắc hạn, không thay phần mềm thuế.

3.19.7 SOCIAL_INSURANCE_MONTHLY — đóng BHXH hằng tháng

Khi có ít nhất một lao động. Hạn ngày cuối tháng đang diễn ra ($\text{due_month_offset} = 0, \text{due_day} = 31$ kếp cuối tháng). Căn cứ Điều 34 Luật BHXH 41/2024/QH15 và Điều 4 Nghị định 58/2020/NĐ-CP (quỹ tai nạn lao động). Tỷ lệ đóng không nằm trong rule; người dùng hỏi tỷ lệ thì đi pipeline RAG, không đi lịch.

3.19.8 LABOUR_REPORT_H1 và H2 — báo cáo tình hình lao động

Hai rule cùng căn cứ Điều 12 Bộ luật Lao động 45/2019/QH14 và Điều 4 Nghị định 145/2020/NĐ-CP, khác tháng: trước 05/06 và trước 05/12. Lọc $\text{min_employees} \geq 1$. Tách H1/H2 thay vì một rule “hai lần/năm” để task hiện thành hai mốc trên lịch tháng, để đánh dấu độc lập.

3.19.9 LABOUR_SAFETY_REPORT — báo cáo an toàn vệ sinh lao động

Khi $\text{min_employees} \geq 10$. Hạn 10 tháng 01 năm sau. Căn cứ Điều 81 Luật An toàn, vệ sinh lao động 84/2015/QH13. Ngưỡng 10 lao động là điều kiện lọc trong mã, phản ánh hướng tập trung

DNNVV có quy mô đủ để nghĩa vụ báo cáo này trở nên thường gặp; khi văn bản đổi ngưỡng phải sửa `applies_to`.

3.19.10 FINANCIAL_STATEMENT — báo cáo tài chính năm

Mọi tổ chức. Hạn mã hoá cuối tháng 3 (90 ngày sau 31/12 với năm dương lịch). Căn cứ Điều 29 Luật Kế toán 88/2015/QH13 và Điều 80 Thông tư 133/2016/TT-BTC (chế độ kế toán DNNVV). Nộp cơ quan thuế và thống kê được nêu trong mô tả; hệ thống không tạo hai task tách cơ quan.

3.19.11 INVOICE_USAGE_CHECK — rà soát hóa đơn điện tử

Rule quý, hạn ngày 15 tháng đầu quý sau: đây là **mốc nội bộ** nhắc rà soát việc lập, gửi, lưu hóa đơn theo Nghị định 123/2020/NĐ-CP Điều 9 và Thông tư 78/2021/TT-BTC Điều 6, không phải một tờ khai riêng có số hiệu thống nhất như GTGT. Mục đích: kéo người dùng từ lịch sang tra cứu/hỏi đáp khi sắp đến kỳ, chứ không tự đối soát dữ liệu tổng cục thuế.

3.19.12 Bảng tổng hợp rule

Bảng 3.15: 12 rule tuân thủ: tần suất và điều kiện lọc.

Mã	Tần suất	Lọc hồ sơ	Hướng hạn trong mã
VAT_MONTHLY	Tháng	Kỳ GTGT tháng	Ngày 20 tháng sau
VAT_QUARTERLY	Quý	Kỳ GTGT quý	Cuối tháng đầu quý sau
CIT_-	Quý	Không	Ngày 30 tháng đầu quý sau
PROVISIONAL			
CIT_-	Năm	Không	Cuối tháng 3
FINALIZATION			
PIT_-	Năm	Có NLĐ	Cuối tháng 3
FINALIZATION			
BUSINESS_-	Năm	Không	30 tháng 01
LICENSE_FEE			
SOCIAL_-	Tháng	Có NLĐ	Cuối tháng
INSURANCE_-			
MONTHLY			
LABOUR_-	Năm	Có NLĐ	05 tháng 6
REPORT_H1			
LABOUR_-	Năm	Có NLĐ	05 tháng 12
REPORT_H2			
LABOUR_-	Năm	≥ 10 NLĐ	10 tháng 01 năm sau
SAFETY_REPORT			
FINANCIAL_-	Năm	Không	Cuối tháng 3
STATEMENT			
INVOICE_-	Quý	Không	Ngày 15 tháng đầu quý sau
USAGE_CHECK			

Người dùng đổi số lao động hoặc kỳ GTGT rồi bấm sinh lại: task tương lai theo rule mới xuất hiện; task quá khứ đã đánh dấu xong giữ nguyên. Không dùng LLM để “đoán” hạn nộp — sai một ngày là sai nghiệp vụ.

3.20 Quy trình chuẩn hóa ngữ liệu

- **Là gì.** Mỗi phần tử JSON là một Điều: số hiệu, chương, số Điều, tiêu đề, nội dung, extra. Manifest liệt kê văn bản được phép nạp.
- **Vì sao dùng.** Cắt theo Điều giữ cách dẫn chiếu của nghề. Cắt chồng theo 512 token phá số Điều. Khóa id ổn định để Postgres và Chroma cùng một mã.
- **Cách dùng.** Script nạp kiểm trường bắt buộc và số hiệu khớp manifest. Điều dài vẫn một vector; Điều ngắn (định nghĩa) vẫn nhúng vì người dùng hay hỏi định nghĩa.
- **Khi nào.** Khi dựng CSDL hoặc khi quản trị nhập file. Script không đánh dấu “còn hiệu lực” — đó là bước biên tập người.

Thứ tự làm việc khi máy trắng: chọn số hiệu trong manifest, dựng `base_data.json` từ bộ Hugging Face, nạp Postgres, rồi mới nhúng vector. Đảo thứ tự (nhúng trước khi có hàng chữ) cho chỉ mục rỗng hoặc lệch id. Nạp chữ và nhúng tách nhau: sau bước Postgres, trang tra cứu đã dùng được, chưa cần khóa mô hình.

Chương 4. THỬ NGHIỆM VÀ ĐÁNH GIÁ

Chương này đánh giá bằng cách chạy hệ thống trên cụm Docker và quan sát bốn nghiệp vụ: hỏi đáp có nguồn, tra cứu không dấu, soát xét hợp đồng, lịch tuân thủ. Phần kiểm thử tự động chưa đưa vào báo cáo. Không công bố F_2 vì chưa có bộ câu gắn nhãn nội bộ.

4.1 Môi trường thử nghiệm

Bảng 4.1: Cấu hình máy và cụm khi chạy thử.

Thành phần	Giá trị
Host	WSL2, Linux
Điều phối	Docker Compose, mạng cầu legal
PostgreSQL	17-alpine, cổng host 23432
Backend	Python 3.10, FastAPI, cổng 8023
Frontend	nginx phục vụ bản Vite, cổng 5173
LLM	gemini-2.5-flash, temperature 0
Embedding	gemini-embedding-001, 1536 chiều
Corpus	38 văn bản, 3.844 Điều

Một phần thử nghiệm cố ý chạy **không** gắn khóa API để chứng minh suy giảm có kiểm soát. Phần hỏi đáp có trích dẫn và soát xét hợp đồng cần khóa; khi hết hạn mức phải ra lỗi rõ, không stack trace.

4.2 Chạy tay trên cụm Docker

Bảng 4.2: Quan sát khi chạy cụm Docker trên máy làm đồ án.

Nhóm	Kết quả	Ý nghĩa
GET /laws	38 văn bản	Catalog khớp manifest
Tìm “thue gtgt”	73 hit	Bỏ dấu hoạt động
Đăng ký DN 12 người, GTGT quý	39 task	Có ATVSLĐ, không VAT tháng
Hỏi đáp không khóa / chưa index	HTTP 503	Suy giảm có kiểm soát
Tải hd.txt	201, ready	Trích chữ lúc tải

Ba lệnh tối thiểu: copy .env ở thư mục gốc (không phải backend/.env), docker compose up -d --build, nạp corpus bằng load_postgres.py.

4.3 Kịch bản hỏi đáp

4.3.1 Câu trong phạm vi corpus

Câu demo: “Công ty mười hai người, nhân viên kinh doanh, thử việc được phép bao lâu?”

- BM25 bắt cụm “thử việc” trong Bộ luật Lao động.
- Vector kéo thêm vài Điều về hợp đồng xác định thời hạn vì gần nghĩa.
- `llm_filter` thường bỏ Điều về hợp đồng mùa vụ.
- Câu trả lời nêu Điều 25: thời gian thử việc theo loại công việc, không phải một con số 60 ngày cho mọi vị trí.
- Citation bấm ra đúng Điều trong kho.

4.3.2 Câu không phải pháp luật

“Hôm nay ăn gì.” Nút `analyze_intent` trả SKIP, không tốn truy hồi. Nếu không có nút này, pipeline đôi khi kéo Điều về an toàn thực phẩm — dễ làm người dùng hiểu nhầm hệ thống đang trả lời ngoài phạm vi.

4.3.3 Câu ngoài corpus

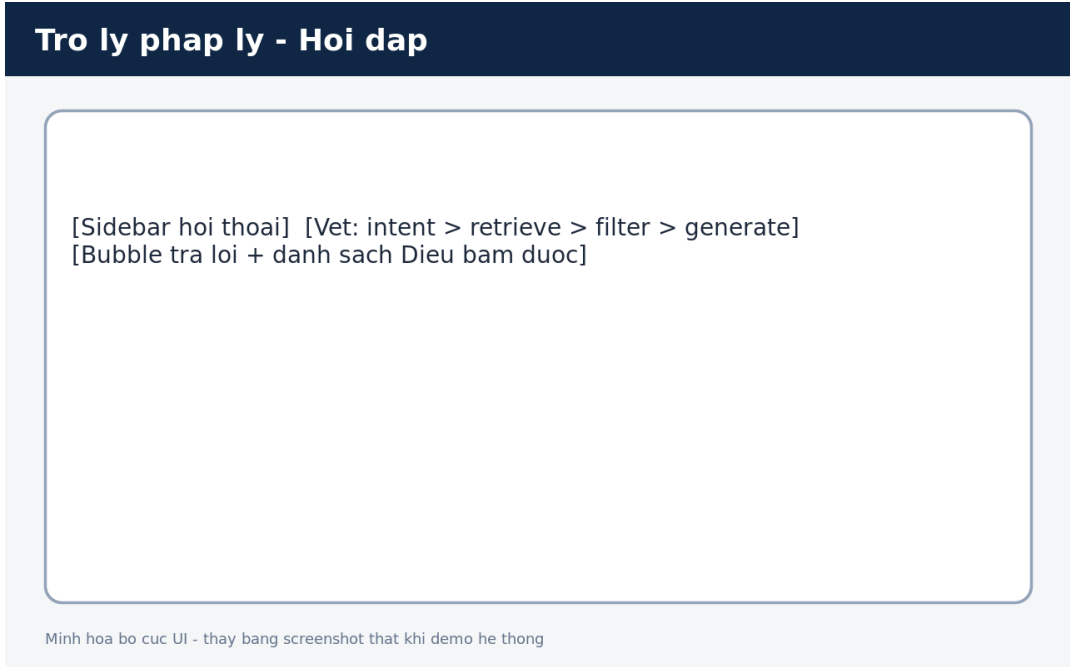
“Thủ tục ly hôn khi vợ chồng đều ở nước ngoài.” Kho không có luật hôn nhân gia đình. Đầu ra phải nói không đủ căn cứ. Nếu mô hình vẫn viết đoạn khuyên chung, danh sách citation sau hậu kiểm phải rỗng.

4.4 Đánh giá module tra cứu và lịch tuân thủ

Bảng 4.3: Đối chiếu hồ sơ doanh nghiệp và task được sinh.

Hồ sơ	Quan sát	Kết luận
12 NLĐ, GTGT quý	39 task; có ATVSLĐ; không VAT tháng	Lọc <code>applies_to</code> đúng
3 NLĐ, GTGT tháng	Có VAT tháng; không ATVSLĐ	Ngưỡng 10 người hoạt động
Đổi quý → tháng rồi sinh lại	Task cũ đã tick không mất	Unique (công ty, rule, kỳ)

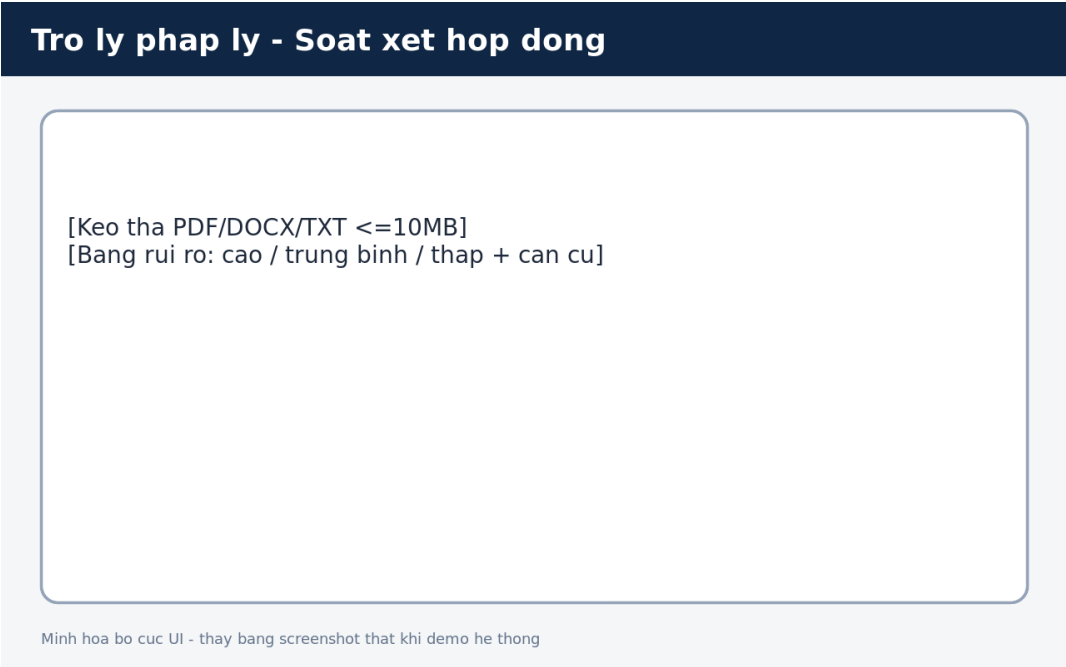
Ô tìm không dấu “thue gtgt” trả 73 dòng; chữ trên trang vẫn còn dấu nhờ bôi đậm phía client. Kết quả này đáp ứng yêu cầu YC-05.



Hình 4.1: Bố cục trang hỏi đáp: cột hội thoại, vết pipeline, câu trả lời và citation.



Hình 4.2: Bố cục trang tra cứu: cây Chương–Điều và tìm không dấu.



Hình 4.3: Bố cục trang hợp đồng: upload và bảng rủi ro.



Hình 4.4: Bố cục trang lịch tuân thủ: ô đếm và nhóm theo tháng.

4.5 Đánh giá giao diện (định tính)

Khi vận hành thử, cột vết pipeline giúp theo dõi tiến độ; nút “Hỏi AI về Điều này” trên trang luật hỗ trợ chuyển từ tra cứu sang hỏi đáp. Trên màn hẹp, thanh hội thoại còn chật — đây là hạn chế giao diện.

4.6 Đối chiếu mục tiêu Chương 1

Bảng 4.4: Đối chiếu bảy mục tiêu cụ thể với bằng chứng.

MT	Nội dung rút gọn	Bằng chứng
1	Corpus cắt theo Điều	3.844 bản ghi, manifest 38 luật
2	Pipeline 7 bước + SSE	LangGraph + vết tiến độ trên UI
3	Tra cứu cây + không dấu	73 hit “thue gtgt”
4	Soát xét hợp đồng nền	Upload 201, poll status
5	Lịch 12 rule	39 task với hồ sơ mẫu
6	JWT + cách ly	404 khi xem chéo hội thoại
7	Đóng gói Docker	Compose 3 dịch vụ

4.7 Tiêu chí đánh giá định tính

Đánh giá sản phẩm khi chạy tay trên Compose theo năm tiêu chí (Bảng 4.2):

1. Dẫn chiếu mở được Điều trong kho, không phải URL ngoại vi bìa.
2. Tra cứu không dấu vẫn đọc được đoạn còn dấu.
3. Suy giảm: thiếu LLM vẫn đăng nhập, đọc luật, xem lịch.
4. Docker một lệnh cho ra đủ ba dịch vụ.
5. Lịch khớp hồ sơ (quý/tháng, có/không lao động).

Không tự gán sao 1–5. Kết luận đạt/không đạt theo quan sát trên máy làm đồ án.

4.7.1 Độ trễ, sẵn sàng và giới hạn đo

- **Là gì.** Quan sát tay: FTS dưới một giây; hỏi đáp vài giây đến hơn mười giây (mạng Gemini + HyDE + filter); soát xét vài trang chạy nền, poll 2,5 giây.
- **Vì sao dừng.** Chứng minh UI không đóng sớm và cụm vẫn sống khi thiếu khóa — không phải benchmark công bố.
- **Cách dùng.** Cố ý không gán khóa: /health 200, login được, GET /laws đủ 38 văn bản, FTS và lịch chạy; chỉ chat/soát xét 503 hoặc failed.

- **Khi nào.** Không công bố “trung bình x giây” (Wi-Fi, hạn mức miễn phí nhiều). Không công bố $P/R/F_2$: chưa gắn nhãn bộ câu nội bộ.

4.8 Kịch bản demo đề xuất khi bảo vệ

Thứ tự dưới đây phủ đủ bốn nghiệp vụ và một nhánh lỗi:

1. Đăng ký doanh nghiệp 12 lao động, GTGT quý; mở lịch, chỉ rule khớp.
2. Tra cứu “thue gtgt” không dấu; mở một Điều; bấm hỏi về Điều đó.
3. Đặt câu “thử việc được bao lâu”; chỉ citation; đối chiếu Điều gốc.
4. Tải hd.txt; đợi soát xét; đọc một finding.
5. (Nếu có) tắt khóa hoặc xóa index tạm — hỏi đáp 503, tra cứu còn.

Kịch bản nhằm chứng minh hệ thống là ứng dụng vận hành được, không phải notebook RAG.

Chương 5. KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Kết quả đạt được

Đồ án đã xây dựng ứng dụng web chạy trên Docker Compose, hỗ trợ nhiều tài khoản và nhiều tổ chức. Người dùng đặt câu hỏi tiếng Việt thông thường, nhận câu trả lời kèm danh sách Điều mở được nguyên văn. Kho 38 văn bản được duyệt theo cây Chương–Điều và tìm được khi thiếu dấu. Chức năng soát xét hợp đồng xuất bảng rủi ro khi có khóa mô hình. Lịch tuân thủ sinh các mốc thuế, bảo hiểm, lao động và BCTC theo hồ sơ từng doanh nghiệp.

Các quyết định kỹ thuật then chốt gồm: cắt ngữ liệu theo Điều; truy hồi lai BM25 và vector; hậu kiểm căn cứ so với tập truy hồi; tách pha tìm không dấu và pha hiển thị còn dấu; dùng cột seq cho thứ tự tin nhắn; trả 404 khi truy cập chéo hội thoại; duy trì đăng nhập và tra cứu khi thiếu khóa API.

Tầng máy chủ chạy trên PostgreSQL.

Đối chiếu bảy mục tiêu cụ thể ở Chương 1: corpus cắt theo Điều đạt ở mức nhập và hiển thị, chưa đạt lịch sử hiệu lực; pipeline bảy bước và SSE đạt trên cấu hình sản phẩm (nút rerank tắt); tra cứu cây và không dấu đạt; soát hợp đồng đạt với tệp có lớp chữ; lịch 12 rule đạt, công thức nằm trong mã; JWT và cách ly đạt; Docker Compose đạt trên môi trường báo cáo. Mục tiêu “mọi căn cứ mở được Điều” đạt khi kho đã nạp và citation đi qua hậu kiểm. Điều kiện đó không biến câu chữ thành tư vấn pháp lý có chữ ký.

Ba chỗ dễ sai đã xử lý trong mã: không tin `created_at` để sắp tin nhắn trong cùng một giao dịch; Compose không đọc `backend/.env`; cùng origin lúc đóng gói làm CORS gần như không chạy — không sao chép * từ máy lập trình sang máy thật.

5.2 Hạn chế

Đồ án không thực hiện OCR: tệp PDF ảnh bị từ chối lúc tải. Không dùng hàng đợi phân tán. Không tự chọn bản luật còn hiệu lực khi hai số hiệu trùng tên khác năm. Chưa đo P/R/F2 trên bộ câu gán nhãn nội bộ. Giao diện chưa tối ưu cho điện thoại. Token lưu tại `localStorage`.

Hậu kiểm chỉ đối chiếu số Điều, không phát hiện diễn giải sai trên Điều đúng. HyDE và lọc LLM tiêu tốn hạn mức dịch vụ; nhúng lại toàn bộ 3.844 Điều trên gói miễn phí dễ gặp mã 429, nên việc nhúng được chia lô và nghỉ dần.

5.3 Hướng phát triển

1. Bổ sung OCR trước khi trích chữ, hoặc từ chối rõ ràng đối với PDF ảnh.
2. Lưu ngày ban hành và hết hiệu lực trên từng Điều, lọc khi truy hồi.

3. Gắn nhãn bộ câu nội bộ và đo P/R/F2 trên cùng corpus.
4. Chuyển soát xét hợp đồng sang tiến trình worker riêng khi tệp dài.
5. Mô hình hóa viện dẫn chéo thành đồ thị phục vụ câu hỏi đa văn bản.
6. Chuyển token sang cookie `HttpOnly`, bổ sung CSP.
7. Tối ưu giao diện cho màn hình hẹp.

5.4 Kết luận

Đề án giải quyết bài toán hỗ trợ DNNVV tra cứu và hỏi đáp pháp lý trong một kho văn bản hẹp. Bốn nghiệp vụ — hỏi đáp có nguồn, đọc cây văn bản, soát xét hợp đồng, nhắc mốc tuân thủ — chạy trên cùng một ứng dụng web đa người dùng. Hệ thống không bao phủ mọi lĩnh vực pháp luật và không thay thế người hành nghề. Nếu người dùng mở được Điều gốc trước khi hành động, sản phẩm đã đáp ứng mục tiêu đã đặt ra.

TÀI LIỆU THAM KHẢO

- [1] Dương Hữu Thành, *Công nghệ phần mềm*, NXB Thông tin và Truyền thông, 2024.
- [2] H. T. Duong, T. P. Bui, D. H. Nguyen, “QLoRA and RAG Application in Building Admissions Support Chatbot: Performance Evaluation and Knowledge Update Ability,” in N. N. Dao, V. Truong Hoang, F. Dornaika (eds.), *Explainable Intelligence in Digital Twins*, EIDT 2025, LNEE vol. 1531, Springer, Singapore, 2026.
- [3] P. Lewis, E. Perez, A. Piktus *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” *NeurIPS*, vol. 33, pp. 9459–9474, 2020.
- [4] V. Karpukhin, B. Oğuz, S. Min *et al.*, “Dense Passage Retrieval for Open-Domain Question Answering,” *Proc. EMNLP*, pp. 6769–6781, 2020.
- [5] S. Robertson, H. Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond,” *Foundations and Trends in Information Retrieval*, vol. 3, no. 4, pp. 333–389, 2009.
- [6] G. V. Cormack, C. L. A. Clarke, S. Buettcher, “Reciprocal Rank Fusion Outperforms Condorcet and Individual Rank Learning Methods,” *Proc. SIGIR*, pp. 758–759, 2009.
- [7] L. Gao, X. Ma, J. Lin, J. Callan, “Precise Zero-Shot Dense Retrieval without Relevance Labels,” *Proc. ACL*, pp. 1762–1777, 2023.
- [8] Quốc hội, *Luật Hỗ trợ doanh nghiệp nhỏ và vừa*, số 04/2017/QH14.
- [9] Quốc hội, *Bộ luật Lao động*, số 45/2019/QH14.
- [10] Quốc hội, *Luật Quản lý thuế*, số 38/2019/QH14.
- [11] PostgreSQL Global Development Group, “Full Text Search,” tài liệu trực tuyến, <https://www.postgresql.org/docs/current/textsearch.html>.
- [12] M. Jones, J. Bradley, N. Sakimura, “JSON Web Token (JWT),” RFC 7519, 2015.
- [13] S. Ramírez, *FastAPI documentation*, tài liệu trực tuyến, <https://fastapi.tiangolo.com>.
- [14] Meta, *React documentation*, tài liệu trực tuyến, <https://react.dev>.
- [15] Docker Inc., *Docker documentation*, tài liệu trực tuyến, <https://docs.docker.com>.
- [16] Chroma, *Chroma documentation*, tài liệu trực tuyến, <https://docs.trychroma.com>.
- [17] R. Fielding, M. Nottingham, J. Reschke, “HTTP Semantics,” RFC 9110, 2022.

- [18] T. Bray, “The JavaScript Object Notation (JSON) Data Interchange Format,” RFC 8259, 2017.
- [19] A. Vaswani, N. Shazeer, N. Parmar *et al.*, “Attention Is All You Need,” *NeurIPS*, 2017.
- [20] A. Biryukov, D. Dinu, D. Khovratovich, “Argon2: the memory-hard function for password hashing and other applications,” winner of the Password Hashing Competition, 2015.
- [21] pdt590, “vietnamese-legal-documents,” Hugging Face Datasets, 2024. <https://huggingface.co/datasets/pdt590/vietnamese-legal-documents>.

PHỤ LỤC

Phụ lục A. Chạy hệ thống trên máy

Sao `.env.example` thành `.env` ở thư mục gốc dự án, không phải trong `backend/`. Compose không đọc `backend/.env`. Điền `LLM_API_KEY` (trang AI Studio của Google) và `JWT_SECRET_KEY` (ví dụ `openssl rand -hex 32`). Chạy:

```
docker compose up -d --build
```

Lần đầu nạp luật:

```
docker compose exec backend python scripts/load_postgres.py--truncate
```

Rồi restart backend. Vào `http://localhost:5173`. Tài khoản đăng ký đầu tiên thành quản trị. Tra cứu dùng được ngay. Hỏi đáp cần đã nhúng vector: vào `/admin/corpus` bấm `reindex`, hoặc đợi startup nếu khóa đã có từ đầu.

Thiếu khóa: web vẫn mở, hỏi đáp trả 503 chữ tiếng Việt, không phải stack trace.

Phụ lục B. Một số đường API

Đường	Việc
POST <code>/api/v1/auth/register</code>	Tạo user, có thể kèm organization
POST <code>/api/v1/auth/login</code>	Cấp access + refresh
POST <code>/api/v1/auth/refresh</code>	Đổi access, kiểm tra type
GET <code>/api/v1/conversations</code>	Danh sách hội thoại của đúng user
POST <code>/api/v1/legal/chat/stream</code>	SSE hỏi đáp
GET <code>/api/v1/laws</code>	Catalog
GET <code>/api/v1/laws/search</code>	FTS
GET <code>/api/v1/laws/{id}/articles</code>	Cây Chương–Điều
POST <code>/api/v1/documents</code>	Tải file
POST <code>/api/v1/documents/{id}/review</code>	Đánh giá, chạy nền
GET <code>/api/v1/compliance/tasks</code>	Lịch
POST <code>/api/v1/admin/corpus/reindex</code>	Nhúng lại, chỉ admin

Phụ lục C. Chạy hàng loạt câu hỏi

Quản trị mở trang `/admin/batch-eval`, gửi một mảng JSON (mỗi phần tử có câu hỏi) để pipeline chạy lần lượt, xem tiến độ và tải kết quả. Chế độ này bỏ qua phân tích ý định (luôn truy hỏi) và không gắn lịch sử hội thoại, nhằm tái lập được trên cùng bộ câu. User thường không gọi

được các đường này.

Phụ lục D. Ghi chú biên dịch báo cáo

File `main.tex` phải chạy XeLaTeX (gói `fontspec`). Trong Cursor hoặc VS Code, biên dịch bằng `recipe xelatex` (hoặc `latexmk -xelatex`). Đầu file có chỉ thị TEX program trở sang xelatex. Ảnh minh họa nằm trong thư mục `Baocao/images/`; không dùng tùy chọn demo của `graphicx` (tùy chọn đó vẽ hình đen đặc).

Phụ lục E. Biến môi trường chính

Biến	Ý nghĩa
LLM_API_KEY	Khóa gọi Gemini (sinh câu)
EMBEDDINGS_API_KEY	Khóa nhúng; có thể trùng khóa LLM
JWT_SECRET_KEY	Khóa ký JWT, tối thiểu đủ dài ngẫu nhiên
LEGAL_DATABASE_URL	Chuỗi kết nối Postgres (Compose tự ghép)
CORS_ALLOW_ORIGINS	Danh sách origin, dùng lúc dev Vite
LLM_MODEL	Mặc định <code>gemini-2.5-flash</code>
EMBEDDINGS_MODEL	Mặc định <code>gemini-embedding-001</code>
POSTGRES_PORT	Mặc định 23432 trên host

Phụ lục F. Cấu trúc thư mục mã nguồn

- `backend/src/routers`: cổng HTTP.
- `backend/src/services/agents`: LangGraph.
- `backend/src/services/vector_store`: BM25, Chroma, hybrid.
- `backend/src/services/legal`: FTS, catalog.
- `backend/src/services/contracts`: trích chữ, soát xét.
- `backend/src/services/compliance`: seed và sinh lịch.
- `backend/src/models`: ORM.
- `backend/alembic`: migration.
- `frontend/src/pages`: trang.
- `frontend/src/components/chat`: hội thoại.

- `corpus/law_manifest.json`: danh mục số hiệu.
- `data/base_data.json`: Điều.
- `Baocao/`: nguồn báo cáo XeLaTeX.

Phụ lục G. Use case còn lại (rút gọn)

UC-01: khách gửi email, mật khẩu, hồ sơ công ty; hệ thống tạo user, có thể gán admin nếu là người đầu; sinh lịch nếu có tổ chức. UC-02: login cấp hai token; refresh đổi access. UC-04: đổi tên hội thoại, xóa mềm hoặc xóa hẳn tùy API; không xem hội thoại người khác. UC-06: ô tìm trên /laws, ba tầng FTS. UC-08: thành viên cùng tổ chức xem finding. UC-10: admin import JSON, reindex. UC-11: admin khóa tài khoản hoặc đổi vai trò. UC-12: admin gửi bộ câu, nhận file kết quả.

Phụ lục H. Thuật ngữ gắn với cách đóng gói đồ án

Container ở đây là một trong ba tiến trình Compose (Postgres, backend, nginx). **Image** là bản đóng gói từng dịch vụ. **Volume** postgres_data, chroma_db, uploads giữ dữ liệu khi tắt cụm. **Proxy** là nginx nhận một cổng rồi chuyển /api vào backend. **Healthcheck** là pg_isready và /health, để backend đợi CSDL sẵn sàng. **Migration** là bản Alembic đổi cột. **Interceptor** là hàm Axios gán token và gỡ JSON khi gửi file. **Splat** là đoạn URL bắt phần còn lại, dùng cho số hiệu có dấu /.

Phụ lục I. Quy trình nạp corpus và lập chỉ mục

Nạp chữ vào Postgres tách khỏi nhúng vector. `scripts/load_postgres.py` đọc JSON Điều, ghi `legal_knowledge_records` (tùy chọn `--truncate`). Generated column tự tính số Điều và tsvector. Tra cứu dùng ngay sau bước này, không cần khóa Gemini.

Nhúng: lúc khởi động hoặc khi quản trị bấm reindex, hệ thống đọc từng Điều, gọi embedding theo lô 64 bản ghi, ghi vào Chroma, rồi ghi manifest (mô hình, số chiều, số vector). Khác mô hình hoặc khác chiều so với manifest: bắt buộc nhúng lại. Thiếu khóa: bỏ qua nhúng, /health vẫn 200.

BM25: tách từ, cache đĩa theo tokenizer. Không nhằm cache BM25 với volume Chroma.

Thứ tự an toàn khi máy trắng: Compose up, đợi health, load_postgres, restart backend, (nếu có khóa) reindex trên admin. Đảo thứ tự (reindex trước khi có hàng Postgres) cho chỉ mục rỗng hoặc lệch id.

Phụ lục J. Ràng buộc đạo đức và trách nhiệm sử dụng

Hệ thống hỗ trợ thông tin trên kho đóng. Không thay thế luật sư, đại lý thuế, hoặc cơ quan nhà nước. Người dùng phải mở Điều gốc trước khi hành động có hậu quả pháp lý. Nhặt ký hội thoại chứa câu hỏi doanh nghiệp — triển khai thật cần chính sách lưu trữ và quyền xóa (đồ án mới có xóa hội thoại theo chủ sở hữu). Không thu thập dữ liệu cá nhân ngoài tài khoản thử nghiệm.